Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»

КАФЕДРА ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных для управления репетиторскими услугами

Студент	**ИУ7-61Б**		**Платонова**
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсового проекта			**Волкова Л.Л.**
		(подпись, дата)	(И.О. Фамилия)
Консультант			
		(подпись, дата)	(И.О. Фамилия)

2025 г.

РЕФЕРАТ

Отчет 70 с., 16 рис., 21 табл., 10 ист., 2 прил.

ОНЛАЙН–ШКОЛА, БАЗА ДАННЫХ, ТРИГГЕРЫ, SQL, GO

Цель работы — разработать базу данных для управления репетиторскими услугами.

Для достижения цели были решены следующие задачи:

— проанализированы существующие методы и подходы к проектированию баз данных в области репетиторских услуг, выбраны оптимальные сущности и их взаимосвязи для эффективного хранения и обработки данных;

— спроектированы основные сущности базы данных, такие как информация о репетиторах, клиентах, занятиях и оплатах, а также установлены соответствующие ограничения целостности данных;

— реализованы механизмы автоматической генерации записей об оплатах при добавлении информации о занятиях, что значительно упрощает и автоматизирует процесс учета финансовых операций;

— разработана ролевая модель для разграничения доступа к данным в системе, что обеспечивает безопасность и удобство работы для пользователей с различными уровнями доступа;

— проведено тестирование функциональности базы данных, включая методы проверки корректности работы приложения, что подтверждает высокую степень работоспособности и стабильности системы;

— выполнено исследование производительности базы данных при увеличении объема данных и числа одновременно выполняемых запросов.

СОДЕРЖАНИЕ

РЕФЕРАТ **4**

ВВЕДЕНИЕ **7**

1 Аналитическая часть **8**

1.1 Анализ предметной области 8

1.2 Сравнительный анализ существующих решений 8

1.3 Формализация и описание пользователей 10

1.4 Формализация и описание информации, подлежащей хранению в проектируемой базе данных 12

1.4.1 Структура базы данных и связи между пользователями . . . 12

1.5 ER–диаграмма сущностей проектируемой базы данных в нотации Чена 13

1.6 Выбор модели базы данных 15

2 Конструкторский раздел **17**

2.1 Диаграмма проектируемой базы данных 17

2.2 Описание сущностей проектируемой базы данных и ограничений целостности 18

2.3 Принципы проектирования 23

2.3.1 Нормализация данных 23

2.3.2 Ограничения целостности 23

2.3.3 Индексация 23

2.4 Описание триггеров 24

2.4.1 Обновление рейтинга репетитора при добавлении отзыва клиента 24

2.4.2 Автоматическое создание платежа после добавления урока . 26

2.4.3 Триггер автоматического создания записи при оплате контракта 27

2.4.4 Триггер обновления рейтинга клиента при добавлении отзыва репетитора 27

2.5 Описание ролевой модели 29

2.6 Вывод 30

3 Технологический раздел 31
3.1 Выбор системы управления базами данных 31
3.2 Выбор инструментов реализации программной части 32
3.3 Реализация функций 33
3.4 Реализация триггеров 33
3.5 Реализация ролевой модели 36
3.6 Тестирование 40
3.7 Реализация индексов 45
3.8 Описание интерфейса доступа к базе данных 46
3.8.1 Веб–интерфейс приложения 47
3.8.2 Возможности неавторизованного пользователя 47
3.8.3 Регистрация и авторизация 47
3.8.4 Кабинет клиента 48
3.8.5 Кабинет репетитора 48
3.8.6 Кабинет модератора 49
3.8.7 Кабинет администратора 49
3.8.8 Система чатов 50

4 Исследовательская часть 58
4.1 Технические характеристики устройства 58
4.2 Влияние индексации на производительность запросов 58
4.3 Выводы 61

ЗАКЛЮЧЕНИЕ 62

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 63

ПРИЛОЖЕНИЕ А 64

ПРИЛОЖЕНИЕ В 65

ВВЕДЕНИЕ

Цель работы — разработать базу данных для управления репетиторскими услугами.

Для достижения поставленной цели необходимо решить следующие задачи.

— провести анализ предметной области репетиторских услуг, определить основные сущности и их взаимосвязи;

— сформулировать требования и ограничения к разрабатываемой базе данных;

— спроектировать сущности базы данных и определить ограничения целостности данных;

— спроектировать триггеры, которые будут автоматически выполнять важные действия при изменениях в базе данных;

— спроектировать ролевую модель на уровне базы данных;

— выбрать средства реализации базы данных и приложения;

— разработать и реализовать сущности базы данных и ограничения целостности;

— описать методы тестирования разработанного функционала и разработать тесты для проверки корректности работы приложения;

— провести исследование производительности базы данных при увеличении объема данных и количества одновременно выполняемых запросов, чтобы оценить её стабильность и эффективность в условиях нагрузки;

1 Аналитическая часть

1.1 Анализ предметной области

Платформа для организации репетиторских услуг представляет собой информационную систему, включающую пользователей различных ролей и компоненты для управления образовательным процессом. В системе принимают участие администраторы, модераторы, репетиторы и заказчики.

Администраторы управляют структурой платформы, назначают модераторов и контролируют работу персонала. Модераторы следят за соблюдением правил и корректностью размещаемого контента, а также могут принимать меры в случае нарушения условий использования. Помимо этого, модераторы обрабатывают заявки, сообщения, конфликтные ситуации и проверяют оплату заказов, обеспечивая надежность финансовых операций.

Репетиторы предлагают образовательные услуги по различным предметам, проводят онлайн–занятия и общаются с заказчиками через систему чатов. Клиенты оставляют заявки на обучение, выбирают подходящих преподавателей, назначают время занятий и оставляют отзывы о пройденных уроках.

Основное назначение платформы — автоматизация процессов подбора преподавателей, планирования занятий, обмена информацией и учета оказанных услуг. Такое программное решение способствует повышению доступности и удобства получения образовательных услуг в онлайн–формате.

1.2 Сравнительный анализ существующих решений

На рынке представлены различные платформы для поиска репетиторов, среди которых рассмотрены следующие:

— skyeng — онлайн-школа, предлагающая занятия с преподавателями через собственную платформу, внимание акцентировано на изучении английского языка [1];

— profi.ru — универсальный сервис поиска специалистов, включая репетиторов, поддерживает онлайн и офлайн форматы общения с заказчиками [2];

— tutorOnline — платформа, ориентированная на онлайн–репетиторство по различным предметам, не имеет встроенного чата [3];

— preply — международная площадка для поиска репетиторов, позволяющая заниматься как онлайн, так и офлайн [4].

Представленные платформы поддерживают механизм поиска, но дополнительные инструменты взаимодействия с пользователями представлены неравномерно. Не все сервисы имеют встроенные чаты для общения между репетиторами и заказчиками, а система отзывов присутствует не во всех решениях.

Проектируемая платформа объединяет преимущества существующих решений, предоставляя пользователям полный набор функций, включая удобный поиск и систему чатов.

В таблице 1 представлен сравнительный анализ существующих решений.

Таблица 1 – Сравнение платформ для онлайн–обучения

Платформа	**Поиск**	**Чаты**	**Отзывы**	**Формат**	**Оплата**	**Личный кабинет**
Skyeng	+	+	+	Только онлайн	Подписка	+
TutorOnline	+	–	+	Только онлайн	Почасовая	+
Preply	+	+	–	Онлайн/ офлайн	Разовая	+
Мой проект	+	+	+	Онлайн/ офлайн	Гибкая	+

Большинство платформ поддерживают поиск репетиторов и систему фильтрации, однако их функциональные возможности различаются.

Например, Skyeng предлагает встроенную систему чатов и отзывов, но работает только по подписке. TutorOnline не имеет встроенного чата, а Preply не предоставляет возможность оставлять отзывы о репетиторах.

Разрабатываемая платформа будет более полноценной для пользователей, так как объединяет сильные стороны конкурентов и устраняет их недостатки — в отличие от других платформ, она предоставляет функциональ-

ность поиска, систему чатов, отзывы, гибкий формат занятий и разнообразные способы оплаты.

1.3 Формализация и описание пользователей

Платформа предназначена для пяти категорий пользователей:

— **гость** (guest) — неавторизованный пользователь, который может просматривать список репетиторов, направления их обучения и отзывы;

— **заказчик** (client) — авторизованный пользователь, который может создавать заказы, взаимодействовать с репетиторами через чаты и оставлять отзывы;

— **репетитор** (repetitor) — пользователь, который просматривает доступные заказы, самостоятельно выбирает подходящие, проводит занятия и общается с заказчиками через встроенные чаты;

— **модератор** (moderator) — пользователь, проверяющий корректность транзакций, модерирующий чаты и имеющий возможность блокировать заказы, нарушающие правила платформы;

— **администратор** (admin) — создает отделы, назначает и увольняет модераторов, устанавливает им уровень зарплаты, следит за нагрузкой, чтобы обеспечить стабильную работу.

На рисунке 1 представлены категории пользователей системы и их основные действия.

Рисунок 1 – Диаграмма прецедентов

1.4 Формализация и описание информации, подлежащей хранению в проектируемой базе данных

Модераторы и отделы в проектируемой базе данных имеют связь «многие ко многим» через таблицу hire_infos. Один модератор может работать в нескольких отделах (например, модерировать заказы и чаты), и у одного отдела может быть несколько модераторов.

Роли пользователей распределены следующим образом:

— администратор создает отделы и назначает модераторов;

— модератор проверяет заказы, чаты и отзывы, также контролирует корректность оплат;

— заказчики и репетиторы взаимодействуют через систему чатов и сами заказы.

1.4.1 Структура базы данных и связи между пользователями

Основные данные в системе организованы следующим образом:

— **пользователи** хранятся в таблицах moderators (модераторы), admins (администраторы), repetitors (репетиторы) и clients (заказчики);

— **отделы** представлены в таблице departments и связаны с модераторами через таблицу hire_infos, что позволяет одному модератору работать сразу в нескольких отделах, руководитель отдела определяется полем departments.head_id;

— **резюме репетиторов** хранится в таблице resumes, где указаны описание, предметы и расценки преподавателя;

— **контракты** фиксируются в таблице contracts и связывают заказчика с репетитором;

— **чаты** создаются в таблице chats, при этом сами сообщения являются отдельной сущностью и хранятся в таблице messages;

— **транзакции оплат** отражаются в таблице transactions, а ожидающие проверки платежи — в pending_contract_payment_transactions;

— **отзывы** располагаются в таблице reviews и привязаны к конкретным контрактам, фиксируя качество работы репетиторов;

— **администраторы** через интерфейс управляют отделами, назначают и удаляют модераторов, а также контролируют работу системы.

1.5 ER–диаграмма сущностей проектируемой базы данных в нотации Чена

Схема «сущность-связь» (также ERD или ER–диаграмма) — это разновидность блок-схемы, где показано, как разные «сущности» (люди, объекты, концепции и так далее) связаны между собой внутри системы. На рисунке 2 представлена ER–диаграмма сущностей проектируемой базы данных.

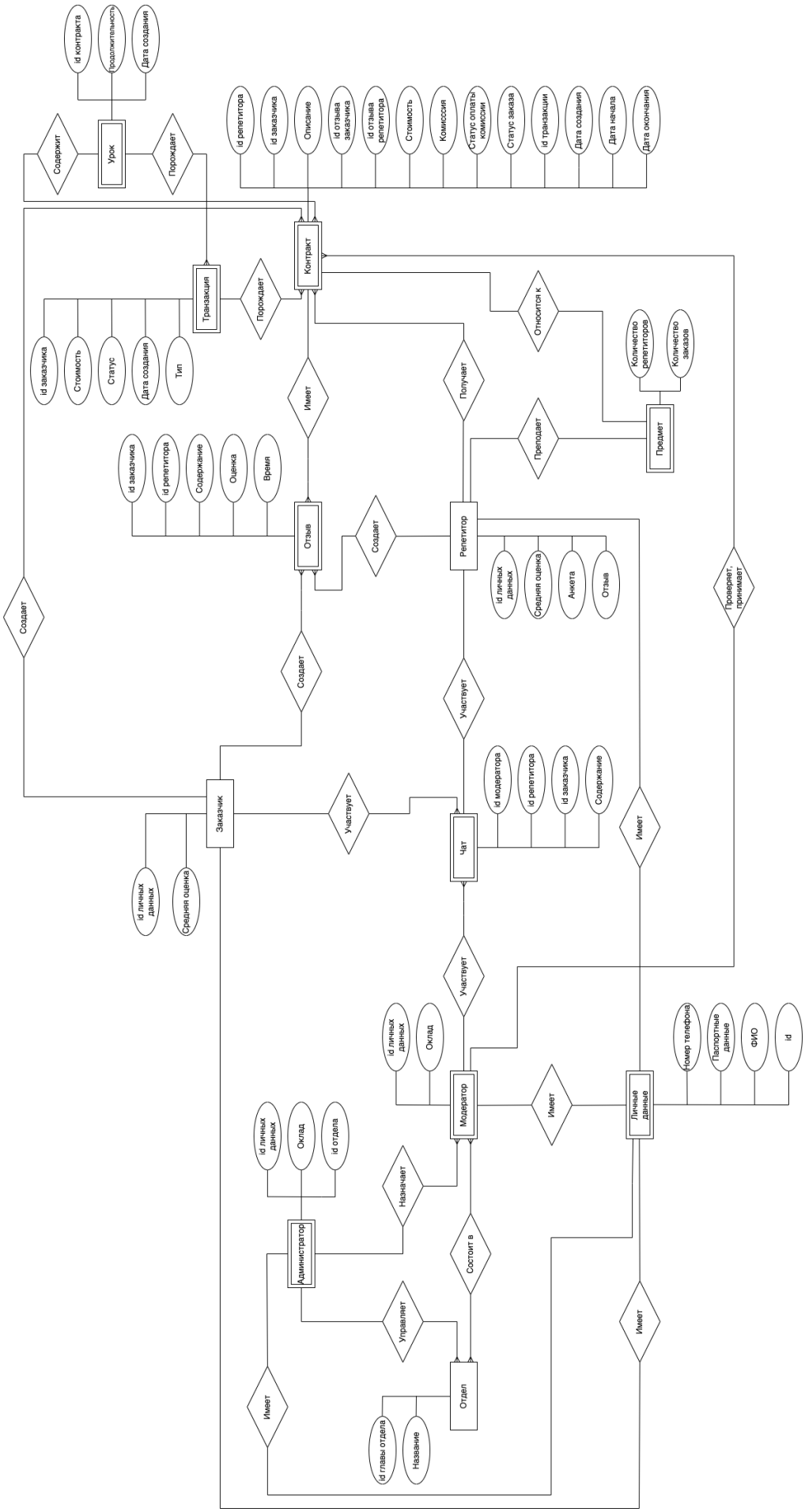

Рисунок 2 – Диаграмма сущность–связь

1.6 Выбор модели базы данных

Для разработки платформы необходимо определить наиболее подходящую модель базы данных. Рассмотрим виды моделей баз данных по отдельности.

— **дореляционные базы данных** — включают иерархические, сетевые модели и структуры, основанные на инвертированных списках. Такие системы не обеспечивают строгую целостность данных и редко применяются в современных веб–приложениях.

— **реляционные базы данных** — организуют хранение данных в виде таблиц с четкими взаимосвязями между сущностями, соблюдая принципы ACID (атомарность, согласованность, изолированность, долговечность). Такой подход гарантирует надежность, структурированность и удобное управление данными.

— **постреляционные базы данных** — расширяют реляционную модель, позволяя хранить сложные типы данных внутри ячеек.

Дореляционные базы данных, такие как иерархические и сетевые, устарели и редко применяются в веб–разработке из–за сложности управления и отсутствия строгих механизмов обеспечения. Постреляционные базы данных предлагают гибкость за счет поддержки сложных структур данных, но они усложняют обработку информации и требуют дополнительных ресурсов.

Для наглядного сравнения различных моделей баз данных рассмотрим их характеристики в таблице 2.

На основе проведенного анализа выбрана реляционная модель базы данных. Данная модель обеспечивает надежное хранение и целостность данных, удобную работу с таблицами, а также простую и понятную реализацию связей между ключевыми сущностями платформы — клиентами, сотрудниками, репетиторами и заказами.

Таблица 2 – Сравнение моделей баз данных

Критерий	**Дореляционные**	**Реляционные**	**Постреляционные**
Структура данных	Иерархическая, сетевая	Табличная с четкими связями	Табличная с вложенными структурами
Целостность данных	Частичная	Высокая (ACID)	Высокая, но сложнее в управлении
Гибкость запросов	Ограниченная	Высокая (SQL)	Высокая, но сложнее в обработке
Поддержка транзакций	Отсутствует или частичная	Полная (ACID)	Полная (ACID)
Сложность реализации	Высокая	Средняя	Высокая

2 Конструкторский раздел

В данном разделе описаны структура базы данных, сущности и их взаимосвязи, а также ограничения для полей.

2.1 Диаграмма проектируемой базы данных

Диаграмма сущностей проектируемой базы данных представлена на рисунке 3.

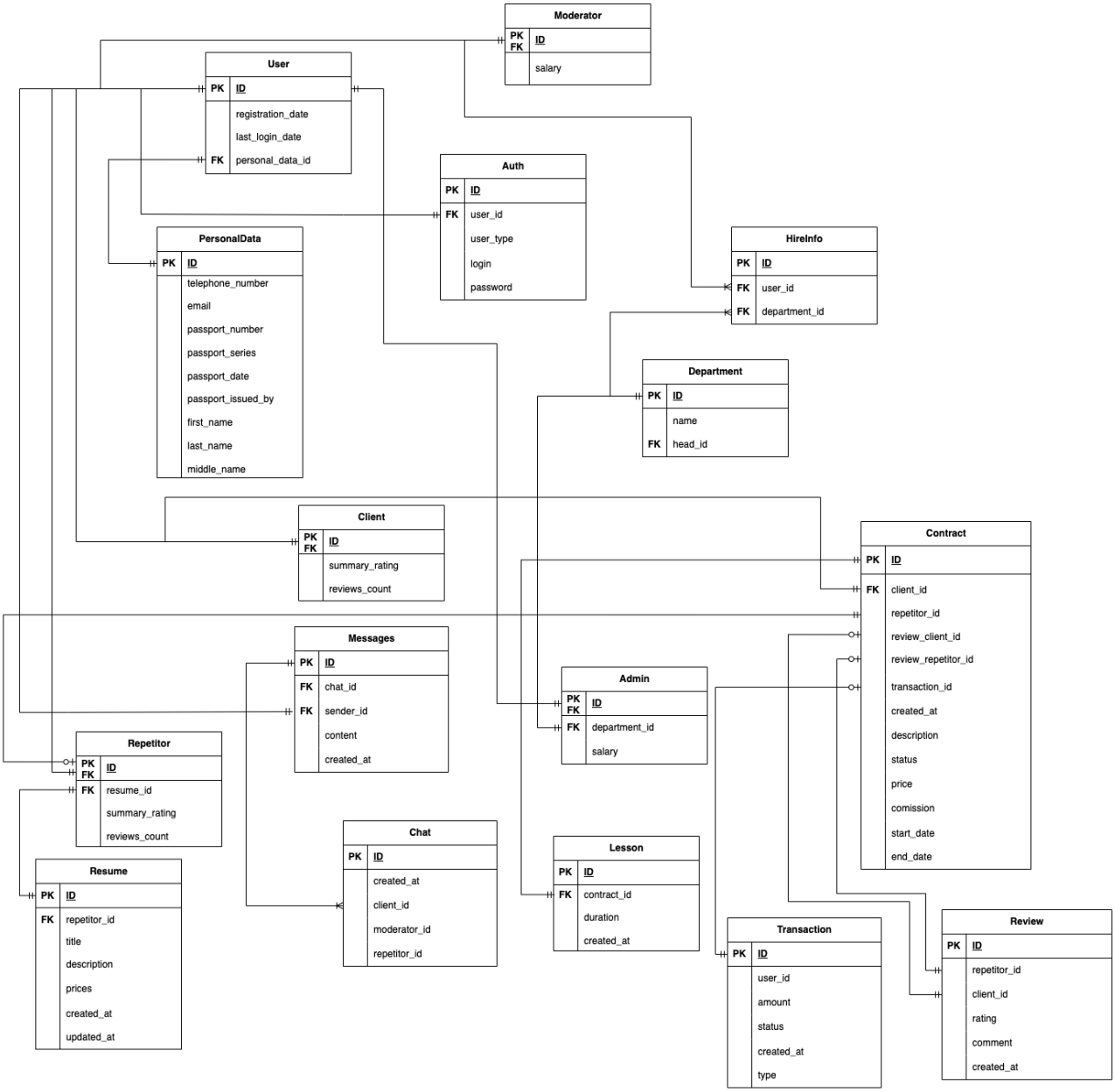

Рисунок 3 – Диаграмма сущностей базы данных

2.2 Описание сущностей проектируемой базы данных и ограничений целостности

Далее представлены типы полей и ограничения целостности базы данных.

— PK (Primary Key) — первичный ключ.

— FK (Foreign Key) — внешний ключ.

— NOT NULL — обязательное поле.

— ограничение уникальности email–адресов пользователей.

— ограничение стоимости заказа в диапазоне 500–10000.

В таблицах 3–15 представлены типы полей, ограничения и описания для основных сущностей базы данных проекта.

Таблица 3 – Поля таблицы Department

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор отдела	PK
Name	Текстовая строка	Название отдела	Обязательное поле
HeadID	Целое число	Руководитель отдела	Обязательное поле, FK → User(ID)

Таблица 4 – Поля таблицы Moderator

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор модератора	PK
Salary	Целое число	Зарплата модератора	Обязательное поле, значение должно быть больше нуля

Таблица 5 – Поля таблицы HireInfo (связь пользователей с отделами)

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор связи	PK
DepartmentID	Целое число	Идентификатор отдела	Обязательное поле, FK → Department(ID)
UserID	Целое число	Идентификатор пользователя	Обязательное поле, FK → User(ID)

Таблица 6 – Поля таблицы User

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор пользователя	PK
PersonalDataID	Целое число	Личные данные пользователя	Обязательное поле, уникальное значение, FK → PersonalData(ID)
RegistrationDate	Дата и время	Дата регистрации пользователя	Обязательное поле
LastLoginDate	Дата и время	Дата последнего входа	Обязательное поле

Таблица 7 – Поля таблицы персональных данных

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор персональных данных	PK
TelephoneNumber	Текстовая строка	Номер телефона	Обязательное поле
Email	Текстовая строка	Электронная почта	Обязательное поле
PassportNumber	Текстовая строка	Номер паспорта	Обязательное поле
PassportSeries	Текстовая строка	Серия паспорта	Обязательное поле
PassportDate	Дата и время	Дата выдачи паспорта	Обязательное поле
PassportIssuedBy	Текстовая строка	Орган, выдавший паспорт	Обязательное поле
FirstName	Текстовая строка	Имя	Обязательное поле
LastName	Текстовая строка	Фамилия	Обязательное поле
MiddleName	Текстовая строка	Отчество	Обязательное поле

Таблица 8 – Поля таблицы Contract

Поле	**Тип**	**Описание**	**Ограничения**
ID	Целое число, автоинкремент	Уникальный идентификатор контракта	PK
ClientID	Целое число	Идентификатор клиента	Обязательное поле, FK → User(ID)
RepetitorID	Целое число	Идентификатор репетитора	Обязательное поле, FK → User(ID)
ReviewClientID	Целое число	Идентификатор отзыва клиента	Обязательное поле, FK → Review(ID)
ReviewRepetitorID	Целое число	Идентификатор отзыва репетитора	Обязательное поле, FK → Review(ID)
TransactionID	Целое число	Идентификатор транзакции	Обязательное поле
CreatedAt	Дата и время	Дата и время создания контракта	Обязательное поле
Description	Текстовая строка	Текстовое описание контракта	Обязательное поле
Status	Целое число	Статус контракта	Обязательное поле
PaymentStatus	Целое число	Статус оплаты	Обязательное поле
Price	Целое число	Стоимость контракта	Обязательное поле, значение должно быть больше нуля
Commission	Целое число	Комиссия платформы	Обязательное поле, значение должно быть больше нуля
StartDate	Дата и время	Дата начала действия контракта	Обязательное поле
EndDate	Дата и время	Дата окончания действия контракта	Обязательное поле

Таблица 9 – Поля таблицы отзывов

Поле	**Описание**	**Ограничения**
ID	Уникальный идентификатор отзыва	PK, serial
ClientID	Идентификатор клиента, оставившего отзыв	FK → Client(ID), обязательное поле
RepetitorID	Идентификатор репетитора, о котором оставлен отзыв	FK → Repetitor(ID), обязательное поле
Rating	Оценка (1–5)	обязательное поле
Comment	Текст отзыва	обязательное поле
CreatedAt	Дата и время создания	обязательное поле

Таблица 10 – Поля таблицы Repetitor

Поле	**Описание**	**Ограничения**
ID	Идентификатор пользователя	PK, FK → User(ID)
ResumeID	Идентификатор резюме	FK → Resume(ID), обязательное поле
SummaryRating	Суммарный рейтинг от отзывов	обязательное поле
ReviewsCount	Количество отзывов	обязательное поле

Таблица 11 – Поля таблицы Client

Поле	**Описание**	**Ограничения**
ID	Идентификатор пользователя	PK, FK → User(ID)
SummaryRating	Суммарный рейтинг от отзывов	обязательное поле
ReviewsCount	Количество отзывов	обязательное поле

Таблица 12 – Поля таблицы Transaction

Поле	**Описание**	**Ограничения**
ID	Уникальный идентификатор транзакции (целое число, автоинкремент)	PK
UserID	Пользователь, совершивший транзакцию	FK → User(ID), обязательное поле
Amount	Сумма транзакции	обязательное поле
Status	Статус транзакции	обязательное поле
CreatedAt	Дата и время создания транзакции	обязательное поле
Type	Тип транзакции	обязательное поле

Таблица 13 – Поля таблицы резюме

Поле	**Описание**	**Ограничения**
ID	Идентификатор резюме (целое число, автоинкремент)	PK
RepetitorID	Идентификатор репетитора	FK → Repetitor(ID), обязательное поле
Title	Заголовок резюме (до 255 символов)	обязательное поле
Description	Описание резюме (произвольной длины)	обязательное поле
CreatedAt	Дата создания резюме	обязательное поле
UpdatedAt	Дата последнего обновления резюме	обязательное поле

Таблица 14 – Поля таблицы Chat

Поле	**Описание**	**Ограничения**
ID	Уникальный идентификатор чата (целое число, автоинкремент)	PK
Name	Название чата (до 255 символов)	обязательное поле
Type	Тип чата	обязательное поле
CreatedAt	Дата создания чата	обязательное поле

Таблица 15 – Поля таблицы Message

Поле	**Описание**	**Ограничения**
ID	Уникальный идентификатор сообщения (целое число, автоинкремент)	PK
ChatID	Идентификатор чата	FK → Chat(ID), обязательное поле
SenderID	Идентификатор пользователя	FK → User(ID), обязательное поле
Text	Текст сообщения	обязательное поле
CreatedAt	Дата и время отправки сообщения	обязательное поле

2.3 Принципы проектирования

2.3.1 Нормализация данных

База данных соответствует третьей нормальной форме (3NF). Ее свойства:

— **устранение избыточности** — связи между модераторами и отделами теперь реализованы через таблицу hire_infos, где модераторы могут быть привязаны к нескольким отделам;

— **целостность связей** — каждая сущность отвечает за свою область (например, резюме репетитора вынесено в таблицу Resume).

2.3.2 Ограничения целостности

Для защиты от некорректных данных используются следующие меры:

— **каскадное удаление** — при удалении чата удаляются все сообщения, при удалении резюме репетитора ссылка в таблице Repetitor обнуляется;

— **проверка значений** — зарплата сотрудника должна быть положительной, рейтинг репетитора ограничен диапазоном от 0 до 5;

— **разделение информации** — персональные данные (ФИО, контакты) хранятся в отдельной таблице PersonalData, связанной с пользователем, а данные для авторизации (логин, хеш пароля) — в отдельной таблице Auth.

2.3.3 Индексация

Индексация выполняется следующим образом:

— по логину — в таблице auth авторизации для быстрого поиска при входе в систему;

— по названию отдела — в таблице departments для эффективной фильтрации по отделам;

— по электронной почте — в таблице personal_data для ускоренного поиска пользователей.

2.4 Описание триггеров

2.4.1 Обновление рейтинга репетитора при добавлении отзыва клиента

Для обеспечения автоматического пересчета рейтинга репетитора при получении нового отзыва в базе данных используется данный триггер [5].

После того как к контракту прикрепляется новый отзыв клиента, необходимо учесть эту оценку в статистике репетитора. Для этого используется триггер, который срабатывает при обновлении поля review_client_id в таблице Contract.

Триггер выполняет следующие действия:

— получает рейтинг из таблицы Review по значению review_client_id, указанному в обновленном контракте;

— увеличивает суммарный рейтинг (summary_rating) репетитора на полученную оценку;

— увеличивает счетчик отзывов (reviews_count) репетитора на единицу.

Таким образом, при добавлении нового отзыва система автоматически обновляет статистику репетитора. Средний рейтинг впоследствии может быть рассчитан как

$$\text{Средний рейтинг} = \frac{\text{summary_rating}}{\text{reviews_count}} \tag{1}$$

Пример: если у репетитора уже было два отзыва на 4 и 5, то значения будут такими: summary_rating = 9, reviews_count = 2. После добавления отзыва на 5, значения станут: summary_rating = 14, reviews_count = 3, средний рейтинг будет равен $14/3 \approx 4.67$.

Использование данного триггера позволяет учитывать отзывы без необходимости дополнительной бизнес–логики в приложении.

На рисунке 4 представлена схема триггера «Обновление среднего рейтинга репетитора».

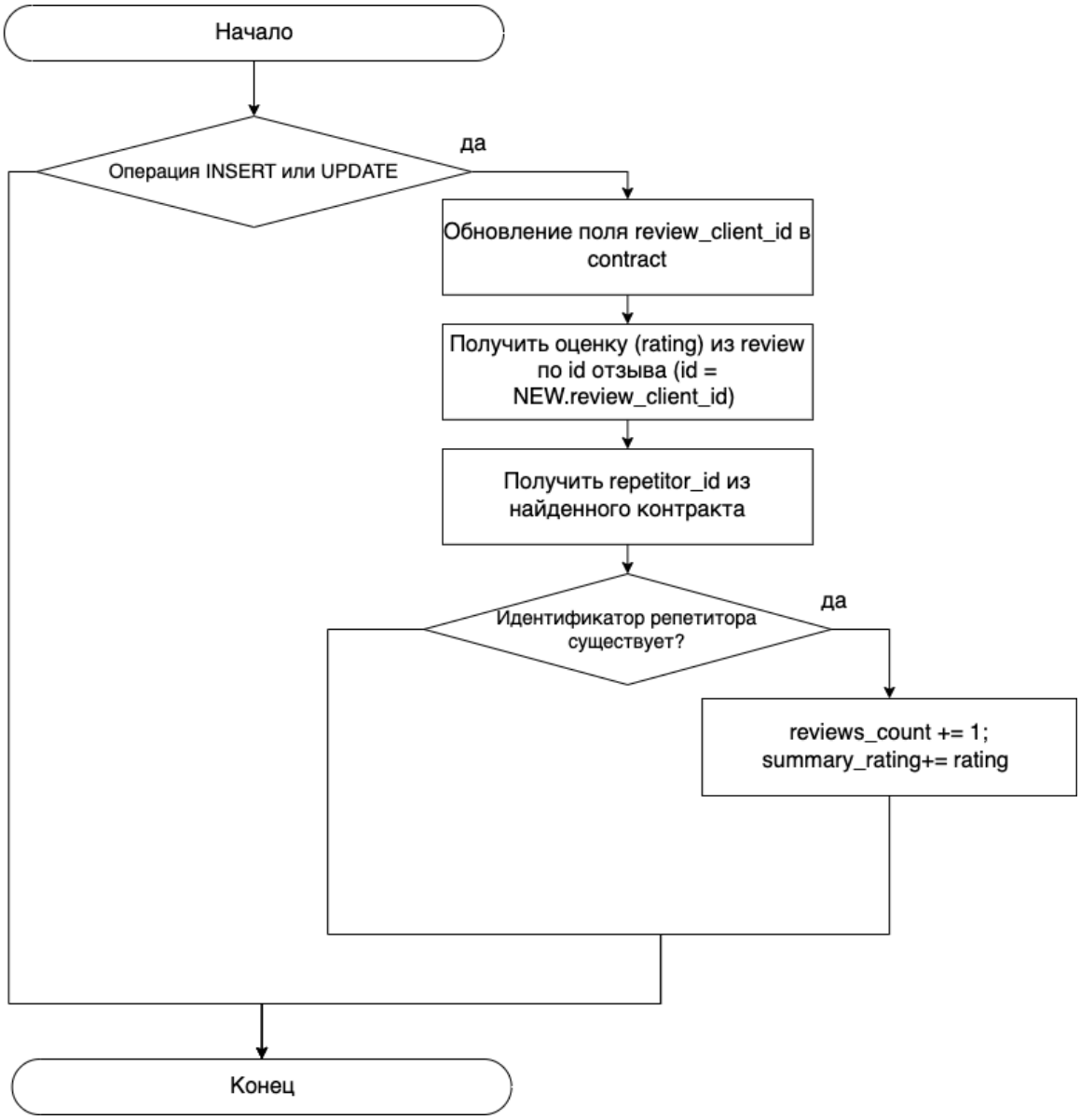

Рисунок 4 – Схема триггера "Обновление среднего рейтинга репетитора"

2.4.2 Автоматическое создание платежа после добавления урока

Для фиксации стоимости каждого проведенного занятия в базе данных используется триггер, который автоматически формирует запись о платеже сразу после добавления новой записи о занятии в таблицу уроков.

После того как в таблицу Lesson добавляется новый урок, триггер получает данные о контракте, к которому относится этот урок, а именно стоимость за час занятий и идентификатор клиента. Далее на основе продолжительности урока вычисляется стоимость оплаты, пропорциональная времени занятия.

Триггер выполняет следующие действия:

— получает из таблицы Contract параметры соответствующего контракта — почасовую ставку (price) и идентификатор клиента (client_id), связанного с уроком;

— рассчитывает сумму платежа, умножая почасовую стоимость на продолжительность урока в минутах и деля результат на 60, то есть, согласно формуле (2)

$$\text{amount} = \frac{\text{price} \times \text{duration}}{60} \tag{2}$$

— добавляет в таблицу Transaction новую запись с указанием:

  — клиента, которому выставляется счет (user_id);

  — рассчитанной суммы оплаты (amount);

  — времени проведения урока (created_at);

  — статуса транзакции, установленного в значение 1 — «ожидает подтверждения»;

  — типа транзакции, равного 2, что означает «оплата занятия».

Таким образом, вся информация об оплате уроков обновляется автоматически при добавлении урока, что упрощает учет финансов, исключая необходимость дополнительной обработки на уровне приложения.

2.4.3 Триггер автоматического создания записи при оплате контракта

Чтобы модераторы могли контролировать транзакции, создаваемые репетиторами, в базе данных реализован специальный триггер [5]. Он автоматически добавляет информацию о новой оплате в таблицу, которую проверяет модератор.

Когда репетитор создает новую транзакцию (например, оплату по контракту), необходимо зафиксировать этот факт отдельно, чтобы модератор мог проверить корректность операции.

Триггер срабатывает после вставки новой записи в таблицу Transaction при выполнении условий:

— статус транзакции равен 1 — это означает, что платеж еще не подтвержден;

— тип транзакции равен 1 — оплата по контракту, а не другой тип перевода.

Если указанные условия выполняются, триггер автоматически создает запись об оплате с идентификатором, пользователем, суммой и датой — в таблице pending_contract_payment_transactions, которую затем просматривает модератор для проверки ожидающих платежей.

Благодаря данному триггеру процесс верификации платежей автоматизирован: как только репетитор создает транзакцию, система самостоятельно формирует запись для модератора, без участия пользователя.

Пример: если репетитор отправил платеж на 1000 рублей, создается новая транзакция со статусом 1 и типом 1. Благодаря триггеру автоматически появляется соответствующая запись, которую затем проверяет модератор в таблице pending_contract_payment_transactions.

2.4.4 Триггер обновления рейтинга клиента при добавлении отзыва репетитора

Для автоматического пересчета рейтинга клиента при добавлении отзыва от репетитора в базе данных реализован триггер под следующим назва-

нием: trigger_update_client_rating [5].

Когда репетитор добавляет отзыв о клиенте в рамках завершенного контракта, его оценка должна быть учтена в статистике клиента. Для этого используется триггер, срабатывающий при обновлении поля review_repetitor_id в таблице Contract.

Триггер выполняет следующие действия:

— извлекает рейтинг, выставленный клиенту, из таблицы Review по значению review_repetitor_id;

— находит клиента, связанного с этим контрактом;

— увеличивает значение summary_rating клиента на полученную оценку;

— увеличивает значение reviews_count на 1.

Средний рейтинг клиента рассчитывается по формуле (1).

Пример: если клиент имел оценки 3 и 5 (то есть значения величин summary_rating = 8, reviews_count = 2), то после получения нового отзыва с оценкой 4 значения обновятся до: summary_rating = 12, reviews_count = 3, и средний рейтинг составит $12/3 = 4.0$.

Благодаря данному триггеру обновление статистики клиента происходит автоматически, без необходимости дополнительной обработки на стороне приложения.

2.5 Описание ролевой модели

В проектируемой базе данных присутствуют следующие роли, соответствующие пользователям из раздела 1.3:

— **Г** — гость;

— **К** — клиент;

— **Р** — репетитор;

— **М** — модератор;

— **А** — администратор.

Права обозначены следующим образом:

— S — чтение;

— I — вставка;

— U — изменение;

— D — удаление.

Таблица 16 – Права ролей базы данных

	Г	**К**	**Р**	**М**	**А**
Репетиторы	S	S	S, I, U	S, I, U	S, I, U, D
Контракты	—	S, I	S, I	S, I, U	S, I, U, D
Направления	S	S	S	S, I, U	S, I, U
Пользователи	S	S	S	S, I, U	S, I, U, D
Персональные данные	—	—	—	S, I, U	S, I, U, D
Аутентификация	—	—	—	—	S, I, U, D
Чаты	—	S, U, I	S, U, I	S, I, U, D	S, I, U, D
Сообщения	—	S, I, U	S, I, U	S, I, U, D	S, I, U, D
Департаменты	—	—	—	S	S, I, U, D
Клиенты	—	S, I, U	S	S, I, U	S, I, U, D
Резюме	—	S	S, I, U	S, I, U	S, I, U, D
Отзывы	S	S, I, U	S, I, U	S, I, U, D	S, I, U, D
Модераторы	—	—	—	S	S, I, U, D
Администраторы	—	—	—	—	S, I, U, D
Транзакции	—	S, I	S, I	S, I, U	S, I, U, D

2.6 Вывод

В данном разделе описаны основные элементы проектирования базы данных для системы онлайн–репетиторства. Рассмотрены структура базы данных, диаграмма сущностей и их взаимосвязи, определяющие роли каждой сущности в системе. Описаны ограничения целостности, включая проверки уникальности, обязательность полей и каскадные операции. Представлены несколько триггеров: пересчет среднего рейтинга репетитора при появлении новых отзывов, автоматическое формирование записей о платежах после добавления уроков, создание записей для контроля платежей модераторами и обновление рейтинга клиента при получении отзывов от репетиторов. Также описана ролевая модель с правами доступа для различных ролей (клиент, репетитор, модератор, администратор) к объектам базы данных.

3 Технологический раздел

3.1 Выбор системы управления базами данных

Для реализации базы данных системы онлайн–репетиторства рассматривались различные системы управления базами данных. Сравнительный анализ представлен в таблице 17, где оценки даны по ключевым критериям.

Таблица 17 – Сравнение СУБД по ключевым характеристикам

СУБД	**К1**	**К2**	**К3**	**К4**
Oracle Database	–	+	+	+
MySQL	+	–	+	+
Microsoft SQL Server	–	+	+	+
PostgreSQL	+	+	+	+

К1 — открытое ПО. PostgreSQL является бесплатной и открытой СУБД, что позволяет избежать лицензионных затрат и кастомизировать систему под нужды проекта.

К2 — поддержка расширений. В PostgreSQL есть встроенные средства для написания функций и триггеров, а также возможность подключать дополнительные модули. Это позволяет автоматизировать работу с базой и добавлять нужный функционал.

К3 — масштабируемость и производительность. PostgreSQL эффективно справляется с большими объемами данных и сложными запросами, что гарантирует хорошую производительность системы при росте.

К4 — сообщество и документация. PostgreSQL имеет обширную документацию, а также доступна подробная документация.

Таким образом, на основе данных критериев выбрана PostgreSQL, потому что она удовлетворяет всем четырем критериям, как система управления базами данных для реализации проекта.

3.2 Выбор инструментов реализации программной части

В качестве языка программирования для реализации серверной части системы выбран Go (Golang). Его использование обусловлено рядом преимуществ:

— **производительность** — golang обеспечивает высокую скорость обработки данных и низкую задержку, что является критически важным для онлайн–систем;

— **совместимость с PostgreSQL** — golang имеет отличную поддержку работы с PostgreSQL через библиотеки database/sql и sqlx;

— **многопоточность** — go поддерживает работу с несколькими потоками с помощью горутин, что позволяет разрабатывать приложения, которые могут выполнять несколько задач одновременно, улучшая производительность и позволяя системе легко масштабироваться.

Для работы с базой данных используется PostgreSQL [6] в сочетании с графическим клиентом DBeaver для наглядного просмотра схемы и выполнения SQL–запросов. Язык Go предоставляет встроенную библиотеку testing, с помощью которой удобно разрабатывать модульные тесты. Совместное применение PostgreSQL, Go и DBeaver [7] обеспечивает при разработке и отладке проекта гибкость и необходимую производительность.

3.3 Реализация функций

Код, демонстрирующий создание основных таблиц и ограничений целостности в базе данных, представлен в листинге 22 приложения 4.3.

3.4 Реализация триггеров

В листинге 1 представлен триггер, который автоматически обновляет суммарный рейтинг и количество отзывов репетитора после изменения идентификатора отзыва клиента в таблице договоров.

Листинг 1 – Триггер, обновляющий рейтинг и количество отзывов репетитора при добавлении отзыва

```
CREATE OR REPLACE FUNCTION update_repetitor_rating_and_count()
RETURNS TRIGGER AS $$
DECLARE
    new_rating INTEGER := 0;
BEGIN
    SELECT rating INTO new_rating
    FROM review_table
    WHERE id = NEW.review_client_id;

    UPDATE repetitor_table rt
    SET
        summary_rating = summary_rating + new_rating,
        reviews_count = reviews_count + 1
    WHERE rt.id = NEW.repetitor_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS update_repetitor_rating_trigger ON
    contract_table;
CREATE TRIGGER update_repetitor_rating_trigger
AFTER UPDATE OF review_client_id ON contract_table
FOR EACH ROW
EXECUTE FUNCTION update_repetitor_rating_and_count();
```

В листинге 2 представлен триггер, срабатывающий при добавлении репетитором новой оплаты. Он автоматически создает запись в таблице для

проверки транзакции модератором, чтобы реализовать процесс верификации перед подтверждением оплаты.

Листинг 2 – Триггер, добавляющий запись в таблицу проверки оплаты при действии репетитора

```
CREATE OR REPLACE FUNCTION
    insert_pending_contract_payment_transaction()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.status = 1 AND NEW.type = 1 THEN
        INSERT INTO pending_contract_payment_transactions (id,
            user_id, amount, created_at)
        VALUES (NEW.id, NEW.user_id, NEW.amount, NEW.created_at);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS after_insert_transaction ON
    transaction_table;
CREATE TRIGGER after_insert_transaction
AFTER INSERT ON transaction_table
FOR EACH ROW
EXECUTE FUNCTION insert_pending_contract_payment_transaction();
```

В листинге 3 представлен триггер, обновляющий рейтинг клиента при добавлении отзыва от репетитора. Он автоматически прибавляет новую оценку к общей сумме и увеличивает счетчик отзывов.

Листинг 3 – Триггер, обновляющий рейтинг клиента при добавлении отзыва от репетитора

```
CREATE OR REPLACE FUNCTION update_client_rating_and_count()
RETURNS TRIGGER AS $$
DECLARE
    new_rating INTEGER := 0;
BEGIN
    SELECT rating INTO new_rating
    FROM review_table
    WHERE id = NEW.review_repetitor_id;
    UPDATE client_table ct
    SET
        summary_rating = summary_rating + new_rating,
```

```
12          reviews_count = reviews_count + 1
13      WHERE ct.id = NEW.client_id;
14      RETURN NEW;
15 END;
16 $$ LANGUAGE plpgsql;
17
18 DROP TRIGGER IF EXISTS update_client_rating_trigger ON
     contract_table;
19 CREATE TRIGGER update_client_rating_trigger
20 AFTER UPDATE OF review_repetitor_id ON contract_table
21 FOR EACH ROW
22 EXECUTE FUNCTION update_client_rating_and_count();
```

В листинге 4 представлен триггер, автоматически создающий транзакцию при добавлении новой записи об уроке. Он извлекает информацию о стоимости занятия и идентификатор клиента из таблицы контрактов, рассчитывает сумму оплаты на основе продолжительности урока и создает соответствующую запись в таблице транзакций.

Листинг 4 – Триггер, автоматически создающий транзакцию при добавлении урока

```
1 CREATE OR REPLACE FUNCTION create_transaction_on_lesson()
2 RETURNS TRIGGER AS $$
3 DECLARE
4      price INTEGER;
5      user_id INTEGER;
6 BEGIN
7      SELECT c.price, c.client_id INTO price, user_id
8      FROM contract_table c
9      WHERE c.id = NEW.contract_id;
10
11     INSERT INTO transaction_table (
12          user_id,
13          amount,
14          created_at,
15          status,
16          type
17     ) VALUES (
18          user_id,
19          (price * NEW.duration)::integer / 60,
20          NEW.created_at,
```

```
21            1,
22            2
23        );
24
25        RETURN NEW;
26 END;
27 $$ LANGUAGE plpgsql;
28
29 DROP TRIGGER IF EXISTS lesson_transaction_trigger ON lesson_table;
30 CREATE TRIGGER lesson_transaction_trigger
31 AFTER INSERT ON lesson_table
32 FOR EACH ROW
33 EXECUTE FUNCTION create_transaction_on_lesson();
```

3.5 Реализация ролевой модели

В данном разделе представлены примеры назначения прав доступа к таблицам базы данных для различных ролей, а также примеры использования этих ролей в запросах.

Для реализации ролевой модели выполняется присвоение прав доступа к таблицам разным ролям: guest, client, repetitor, moderator, admin. Код, демонстрирующий назначение прав доступа ролям, представлен в листинге 23 приложения 4.3.

Пояснение:

— роли moderator и admin обладают расширенными правами на основные таблицы, включая изменения и удаление;

— роль client имеет ограниченный доступ, например, к просмотру и изменению собственных данных;

— роль admin имеет полный контроль, включая удаление данных;

— роль guest имеет только права на чтение или совсем не имеет доступа.

Примеры использования ролей

Для выполнения запросов с учетом ролей в PostgreSQL используется команда SET ROLE <role> внутри транзакции. Данная команда переключает

текущую роль на заданную, что позволяет применять соответствующие права доступа. После выполнения запросов роль сбрасывается командой RESET ROLE.

Ниже приведены примеры реализации методов репозитория, демонстрирующие применение разных ролей.

Листинг 5 – Пример использования роли repetitor

```
1  func (r *ContractRepository) GetContractsByRepetitorID(
2      repetitorID int64, from int64, size int64, status types.ContractStatus,
3  ) ([]types.Contract, error) {
4      tx, err := r.db.Begin()
5      if err != nil {
6          return nil, err
7      }
8      defer tx.Rollback()
9      _, err = tx.Exec("SET ROLE repetitor")
10     if err != nil {
11         return nil, err
12     }
13     query := `
14     SELECT id, client_id, repetitor_id, created_at, description, status, payment_status,
15             review_client_id, review_repetitor_id, price, commission, start_date, end_date
16     FROM ` + r.contractTable + `
17     WHERE repetitor_id = $1 AND status = $2
18     ORDER BY created_at DESC LIMIT $3 OFFSET $4
19     `
20     rows, err := tx.Query(query, repetitorID, status, size, from)
21     if err != nil {
22         return nil, err
23     }
24     defer rows.Close()
25     var contracts []types.Contract
26     for rows.Next() {
27         var c types.Contract
28         if err := rows.Scan(&c.ID, &c.ClientID, &c.RepetitorID, &c.CreatedAt,
       &c.Description,
29             &c.Status, &c.PaymentStatus, &c.ReviewClientID, &c.ReviewRepetitorID,
30             &c.Price, &c.Commission, &c.StartDate, &c.EndDate); err != nil {
31             return nil, err
32         }
33         contracts = append(contracts, c)
```

```
34     }
35     if err := rows.Err(); err != nil {
36         return nil, err
37     }
38     _, err = tx.Exec("RESET ROLE")
39     if err != nil {
40         return nil, err
41     }
42     return contracts, tx.Commit()
43 }
```

Листинг 6 – Пример использования роли client

```
 1 func (r *ContractRepository) GetContractsByClientID(
 2     clientID int64, from int64, size int64, status types.ContractStatus,
 3 ) ([]types.Contract, error) {
 4     tx, err := r.db.Begin()
 5     if err != nil {
 6         return nil, err
 7     }
 8     defer tx.Rollback()
 9     _, err = tx.Exec("SET ROLE client")
10     if err != nil {
11         return nil, err
12     }
13     query := `
14     SELECT id, client_id, repetitor_id, created_at, description, status, payment_status,
15             review_client_id, review_repetitor_id, price, commission, start_date, end_date
16     FROM ` + r.contractTable + `
17     WHERE client_id = $1 AND status = $2
18     ORDER BY created_at DESC LIMIT $3 OFFSET $4 `
19     rows, err := tx.Query(query, clientID, status, size, from)
20     if err != nil {
21         return nil, err
22     }
23     defer rows.Close()
24     var contracts []types.Contract
25     for rows.Next() {
26         var c types.Contract
27         if err := rows.Scan(&c.ID, &c.ClientID, &c.RepetitorID, &c.CreatedAt,
       &c.Description,
28             &c.Status, &c.PaymentStatus, &c.ReviewClientID, &c.ReviewRepetitorID,
```

```
29            &c.Price, &c.Commission, &c.StartDate, &c.EndDate); err != nil {
30            return nil, err
31        }
32        contracts = append(contracts, c)
33    }
34    if err := rows.Err(); err != nil {
35        return nil, err
36    }
37    _, err = tx.Exec("RESET ROLE")
38    if err != nil {
39        return nil, err
40    }
41    return contracts, tx.Commit()
42 }
```

Листинг 7 – Пример использования роли admin для обновления данных модератора

```
1  func (r *ModeratorRepository) UpdateModeratorSalary(moderatorID int64, salary int64)
       error {
2      tx, err := r.db.Begin()
3      if err != nil {
4          return err
5      }
6      defer tx.Rollback()
7      _, err = tx.Exec("SET ROLE admin")
8      if err != nil {
9          return err
10     }
11     query := ‘
12     UPDATE ‘ + r.moderatorTable + ‘ SET salary = $1 WHERE id = $2
13     ‘
14     _, err = tx.Exec(query, salary, moderatorID)
15     if err != nil {
16         return err
17     }
18     _, err = tx.Exec("RESET ROLE")
19     if err != nil {
20         return err
21     }
22     return tx.Commit()
23 }
```

Использование ролей вместе с командами SET ROLE и RESET ROLE обеспечивает эффективное разграничение доступа к данным на уровне базы данных.

3.6 Тестирование

Для проверки корректности работы триггеров и функций базы данных использовалась методология черного ящика, при которой тестирование осуществляется без знания внутренней реализации логики триггеров. Также применялись элементы модульного тестирования и интеграционного тестирования [8] для анализа взаимодействия между различными частями системы: транзакциями, модераторскими действиями и подтверждением оплат.

На рисунке 5 представлена схема тестирования триггеров и функций базы данных.

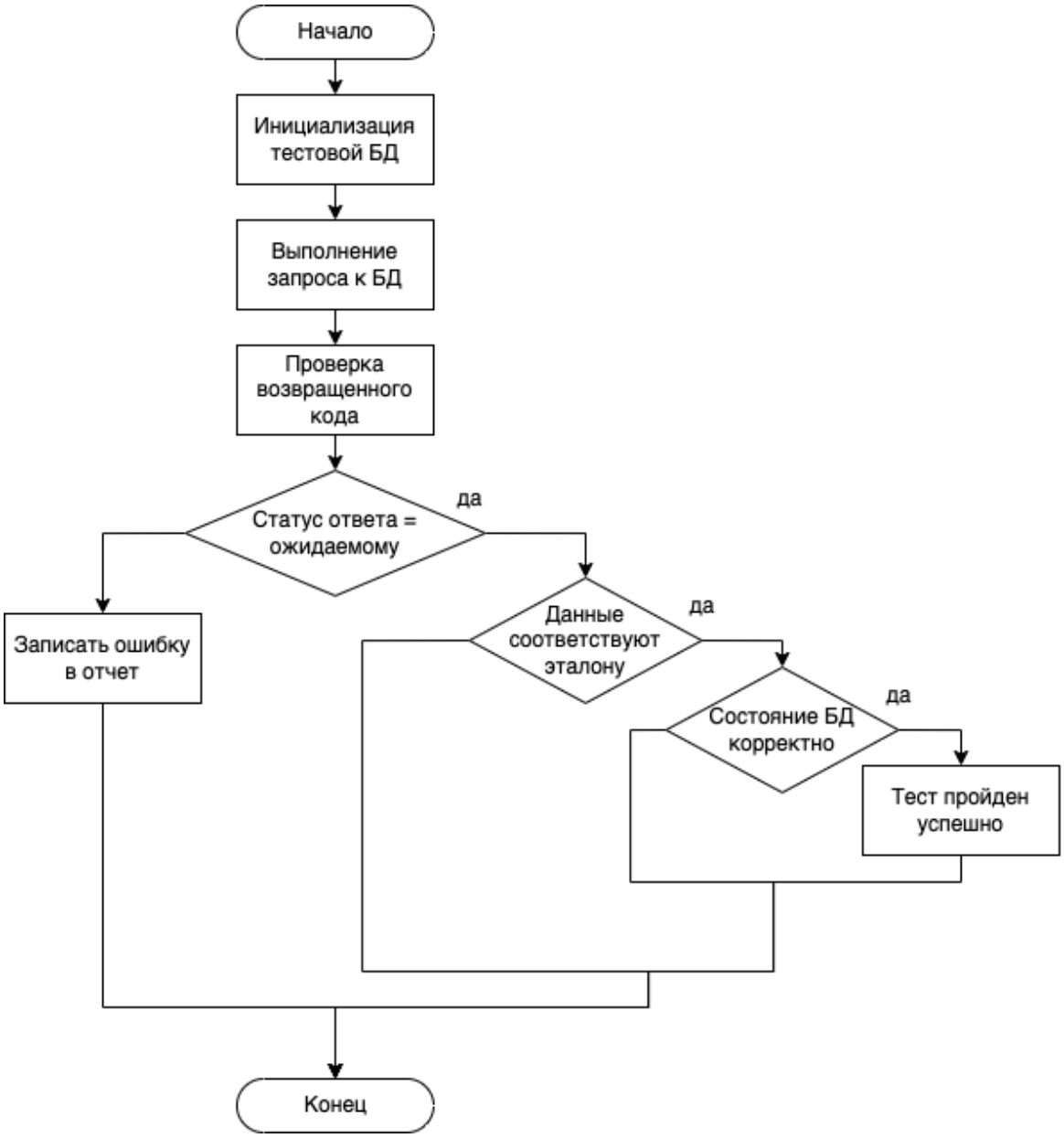

Рисунок 5 – Схема тестирования триггеров и функций базы данных

В таблицах 18—21 представлены тестовые сценарии для четырех триггеров.

Таблица 18 – Тестовые сценарии для триггера создания записи проверки оплаты при добавлении транзакции репетитора

Наименование сценария	**Условие выполнения и ожидаемое поведение**
Успешное создание записи	После вставки транзакции при status = 1 и type = 1 создается новая запись в таблице pending_contract_payment_transactions с правильными данными.
Транзакция с другим статусом	При вставке транзакции со статусом, отличным от 1, запись проверки не создается.
Транзакция с другим типом	При вставке транзакции с типом, отличным от 1, запись проверки не создается.
Отсутствие обязательных полей	Если обязательные поля (например, сумма или user_id) отсутствуют или равны NULL, запись не создается.
Дублирование записей	При повторном добавлении одинаковой транзакции создается запись проверки. Для предотвращения дублей необходима дополнительная логика вне триггера.

Таблица 19 – Тестовые сценарии для триггера автоматического создания платежа после добавления урока

Наименование сценария	**Условие выполнения и ожидаемое поведение**
Успешное создание платежа	После вставки новой записи о занятии в таблицу Lesson создается корректная запись в таблице Transaction с правильным user_id, рассчитанной суммой amount, временем created_at, статусом 1 и типом 2.
Отсутствие контракта	Если для указанного в уроке contract_id отсутствует запись в таблице Contract, платеж не создается, возможна ошибка или откат транзакции.
Нулевая или отрицательная длительность урока	При добавлении урока с длительностью duration меньше или равной нулю платеж не создается, либо сумма равна нулю.
Отсутствие почасовой ставки	Если в контракте отсутствует или равна NULL почасовая ставка price, сумма платежа не рассчитывается, и запись в Transaction не создается.
Корректность расчета суммы	Проверяется, что сумма оплаты соответствует формуле (2) и округляется корректно.
Дублирование платежей	При многократном добавлении одинаковых записей уроков создаются отдельные транзакции, что может требовать дополнительной логики для предотвращения дубликатов.

Таблица 20 – Тестовые сценарии для триггера обновления рейтинга репетитора при добавлении отзыва

Наименование сценария	**Условие выполнения и ожидаемое поведение**
Обновление рейтинга	При обновлении поля review_client_id в таблице contract_table рейтинг репетитора увеличивается на значение рейтинга из отзыва, счетчик отзывов увеличивается на 1.
Отсутствие связанного отзыва	Если значение review_client_id не существует в таблице отзывов, обновление рейтинга не происходит.
Повторное обновление	При повторном обновлении с тем же review_client_id рейтинг увеличивается повторно, что может привести к некорректной статистике без дополнительной логики приложения.
Обновление без изменения поля	При обновлении других полей таблицы contract_table триггер не срабатывает.

Таблица 21 – Тестовые сценарии для триггера обновления рейтинга клиента при добавлении отзыва

Наименование сценария	**Условие выполнения и ожидаемое поведение**
Обновление рейтинга клиента	При обновлении поля review_repetitor_id в таблице contract_table рейтинг клиента увеличивается на значение рейтинга из отзыва, счетчик отзывов увеличивается на 1.
Отсутствие связанного отзыва	Если review_repetitor_id отсутствует в таблице отзывов, обновление не происходит.
Повторное обновление	Повторное обновление с тем же review_repetitor_id приведет к повторному увеличению рейтинга.
Обновление других полей	При обновлении других полей таблицы contract_table триггер не срабатывает.

3.7 Реализация индексов

Для повышения эффективности выполнения авторизационных запросов был добавлен индекс на поле login в таблице auth. Это поле используется при проверке учетных данных пользователей, и индексация позволяет существенно сократить время выполнения таких запросов. Пример исследуемого запроса приведен в листинге 8.

Листинг 8 – Пример авторизационного запроса

```
1 SELECT id
2 FROM auth
3 WHERE login = 'user123';
```

Индекс обеспечивает быстрый доступ к строкам с нужным логином, ускоряя выполнение авторизационных запросов. Для проверки ускорения выполнения запросов после добавления индекса предусмотрена функция генерации пользователей, представленная в листинге 9, которая позволяет провести исследование с заданным количеством пользователей. Эта функция выполняет серию запросов авторизации и измеряет время их выполнения до и после применения индекса.

Листинг 9 – Функция генерации пользователей для тестирования

```
1  func GenerateUsers(size int64, serviceModule *service_logic.ServiceModule) {
2    for i := int64(0); i < size; i++ {
3      initClient := types.InitClientData{
4        InitUserData: types.InitUserData{
5          PersonalData: types.PersonalData{
6            FirstName:       fmt.Sprintf("user%d", i),
7            LastName:        fmt.Sprintf("user%d", i),
8            Email:           fmt.Sprintf("user%d@example.com", i),
9            TelephoneNumber: fmt.Sprintf("+7999999999%d", i),
10         },
11         AuthData: types.AuthData{
12           Login:    fmt.Sprintf("user%d", i),
13           Password: "password",
14         },
15       },
16     }
17     serviceModule.ClientService.CreateClient(initClient) } }
```

В листинге 10 показана функция, которая проверяет скорость выполнения авторизационных запросов до и после добавления индекса, используя заданное количество сгенерированных пользователей.

Листинг 10 – Функция проведения теста производительности по заданному объему пользователей

```
1  func BenchMark(size int64, serviceModule *service_logic.ServiceModule, db *sql.DB) {
2    GenerateUsers(size, serviceModule)
3    start := time.Now()
4    for i := int64(0); i < size; i++ {
5      generateUserIndex := rand.Intn(int(size))
6      serviceModule.AuthService.Authorize(types.AuthData{
7        Login:    fmt.Sprintf("user%d", generateUserIndex),
8        Password: "password",
9      })
10   }
11   elapsed := time.Since(start)
12   fmt.Printf("Time taken without index with %d users: %d ms\n", size,
       elapsed.Milliseconds())
13   data_base.ApplyAuthIndex(db)
14   start = time.Now()
15   for i := int64(0); i < size; i++ {
16     generateUserIndex := rand.Intn(int(size))
17     serviceModule.AuthService.Authorize(types.AuthData{
18       Login:    fmt.Sprintf("user%d", generateUserIndex),
19       Password: "password",
20     })
21   }
22   elapsed = time.Since(start)
23   fmt.Printf("Time taken with index with %d users: %d ms\n", size,
       elapsed.Milliseconds())
24 }
```

3.8 Описание интерфейса доступа к базе данных

Реализован интерфейс как для консольного, так и для веб–приложения. Для консольного интерфейса создано главное меню, которое позволяет пользователю выбрать нужную функциональность, такую как регистрация пользователя, работа с контрактами, отделами, транзакциями.

В листинге 11 представлено меню пользователя, которое появляется

при запуске программы.

Листинг 11 – Консольное меню приложения

```
Main menu
1. User registration
2. Contract functionality
3. Department functionality
4. Transaction functionality
5. Chat functionality
6. Admin functionality
7. Client functionality
8. Repetitor functionality
9. Moderator functionality
10. Bench mark
11. Exit
```

3.8.1 Веб–интерфейс приложения

Для работы с системой разработано веб–приложение [9]. Оно реализует весь функционал, предусмотренный в консольной версии приложения, но с использованием графического интерфейса. Через данное приложение пользователи могут пройти регистрацию, авторизоваться в роли клиента, репетитора, модератора или администратора. Доступный функционал зависит от статуса авторизации и пользовательской роли.

3.8.2 Возможности неавторизованного пользователя

Пользователь, не прошедший авторизацию, имеет ограниченный доступ к функциональности платформы. Он может ознакомиться с основной информацией о платформе, увидеть перечень доступных ролей и перейти к регистрации или авторизации. Любые остальные разделы и операции становятся доступны лишь после входа в систему.

3.8.3 Регистрация и авторизация

Начальным шагом взаимодействия с системой является регистрация, где пользователь выбирает одну из ролей. Далее осуществляется переход на

страницу авторизации. В ней можно ввести логин и пароль, а также выбрать роль для доступа к соответствующему кабинету. После успешного входа пользователю предоставляется доступ к личному кабинету, соответствующему его роли. Маршрутизация реализована в зависимости от роли пользователя и осуществляется автоматически.

3.8.4 Кабинет клиента

Авторизированный клиент получает доступ к личному кабинету, который предоставляет следующие возможности:

— оформление заявок на заключение контрактов с указанием предмета, расписания и предполагаемой стоимости;

— отслеживание статуса поданных заявок — *на рассмотрении*, *активные*, *завершенные*, *отмененные*, *заблокированные*;

— публикация отзывов о репетиторах по завершенным контрактам;

— оплата действующего контракта;

— переход в чат с репетитором для общения;

— просмотр истории всех заявок, платежей и отзывов.

3.8.5 Кабинет репетитора

После регистрации и входа в систему репетитор получает доступ к личному кабинету, в рамках которого доступны следующие функции:

— просмотр актуальных заказов, размещенных клиентами;

— отклик на интересующие контракты с последующим изменением их статуса на *активный*;

— контроль за контрактами в различных статусах;

— публикация отзывов о клиентах по завершенным контрактам;

— оплата действующего контракта;

— досрочное расторжение контракта;

— подтверждение успешного завершения обучения;

— просмотр информации о комиссии, удерживаемой платформой.

3.8.6 Кабинет модератора

После входа в систему модератор видит панель проверки операций и контрактов, где доступны следующие действия:

— просмотр заявок и платежей, ожидающих модерации;

— принятие решений по контрактам и транзакциям (одобрение или отклонение);

— переход в чат с клиентом или репетитором;

— возможность заблокировать контракт при нарушении правил платформы.

3.8.7 Кабинет администратора

Администратор платформы обладает наибольшими полномочиями и может выполнять следующие действия:

— создание и удаление отделов (департаментов) внутри платформы;

— мониторинг и контроль деятельности всех категорий пользователей;

— изменение заработной платы модератора;

— возможность удаления модератора из отдела;

— блокировка учетных записей при выявлении нарушений;

— просмотр ключевых метрик, статистики и аналитических данных по платформе.

3.8.8 Система чатов

Для обеспечения коммуникации между пользователями предусмотрена система чатов. Это отдельный компонент интерфейса, который доступен в личном кабинете любого авторизованного пользователя через соответствующую кнопку профиля. Чат позволяет пользователям обмениваться сообщениями в рамках активных контрактов, чтобы обсудить детали занятий и решить организационные вопросы.

Таким образом, коммуникация встроена в общий рабочий процесс платформы: от регистрации и заключения контракта до его исполнения и контроля. Интерфейс адаптируется в зависимости от роли пользователя.

На рисунках 6–15 представлены страницы интерфейса:

— форма входа в систему (рисунок 6);

— форма регистрации (рисунок 7);

— главная страница приложения (рисунок 8);

— интерфейс чатов (рисунок 9);

— личный кабинет клиента (рисунок 10);

— личный кабинет репетитора (рисунок 11);

— кабинет модератора для подтверждения контрактов (рисунок 12);

— отзыв в кабинете клиента (рисунок 13);

— кабинет репетитора с завершённым заказом (рисунок 14);

— панель администратора (рисунок 15).

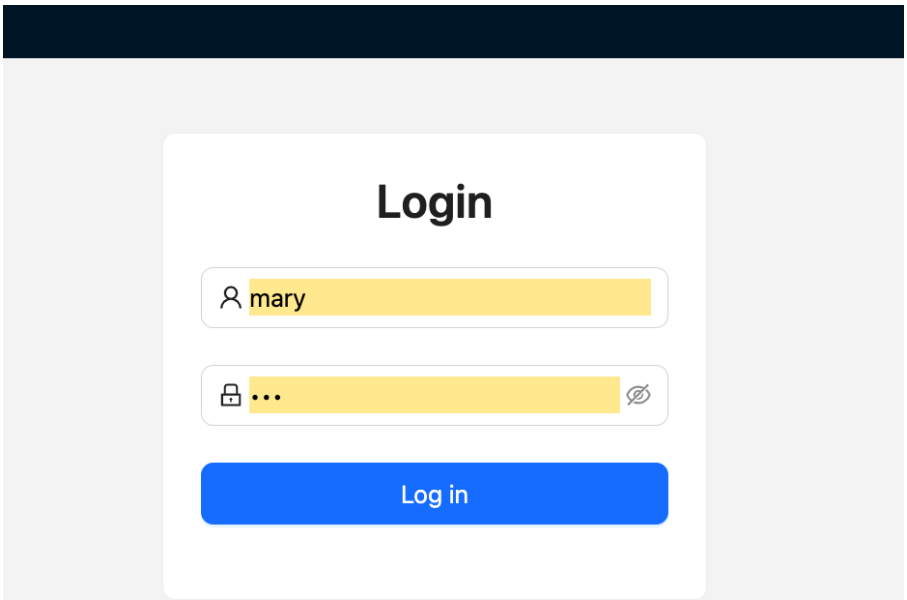

Рисунок 6 – Страница авторизации

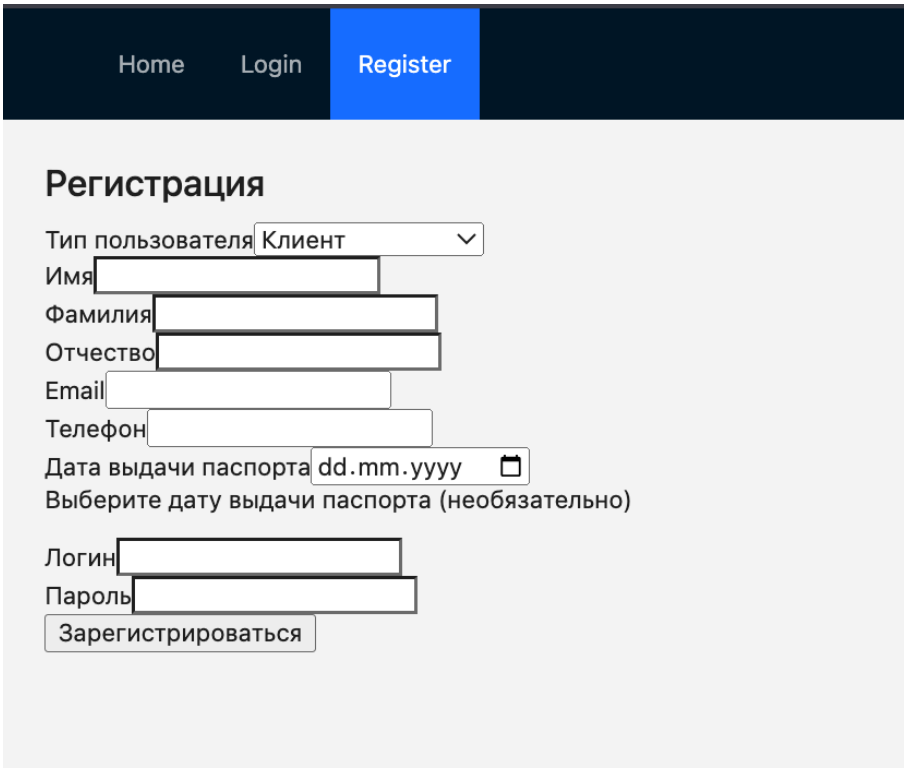

Рисунок 7 – Страница регистрации

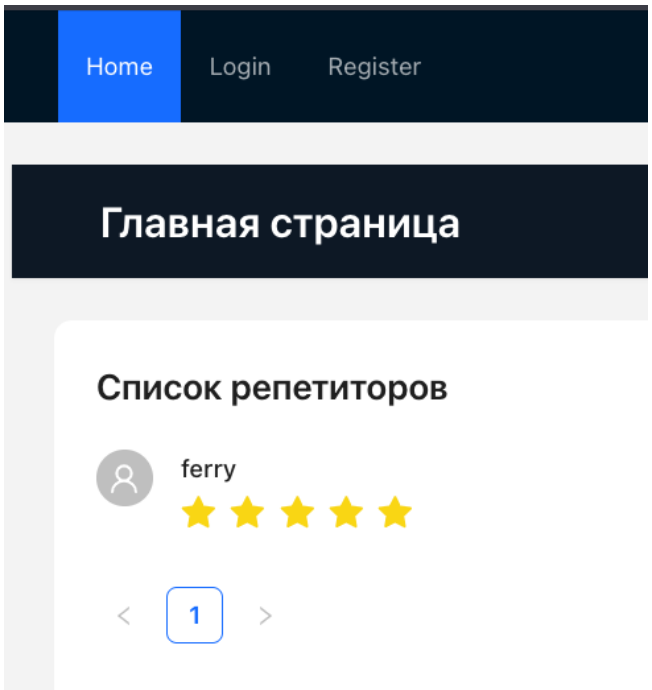

Рисунок 8 – Главная страница сайта

Рисунок 9 – Интерфейс чатов

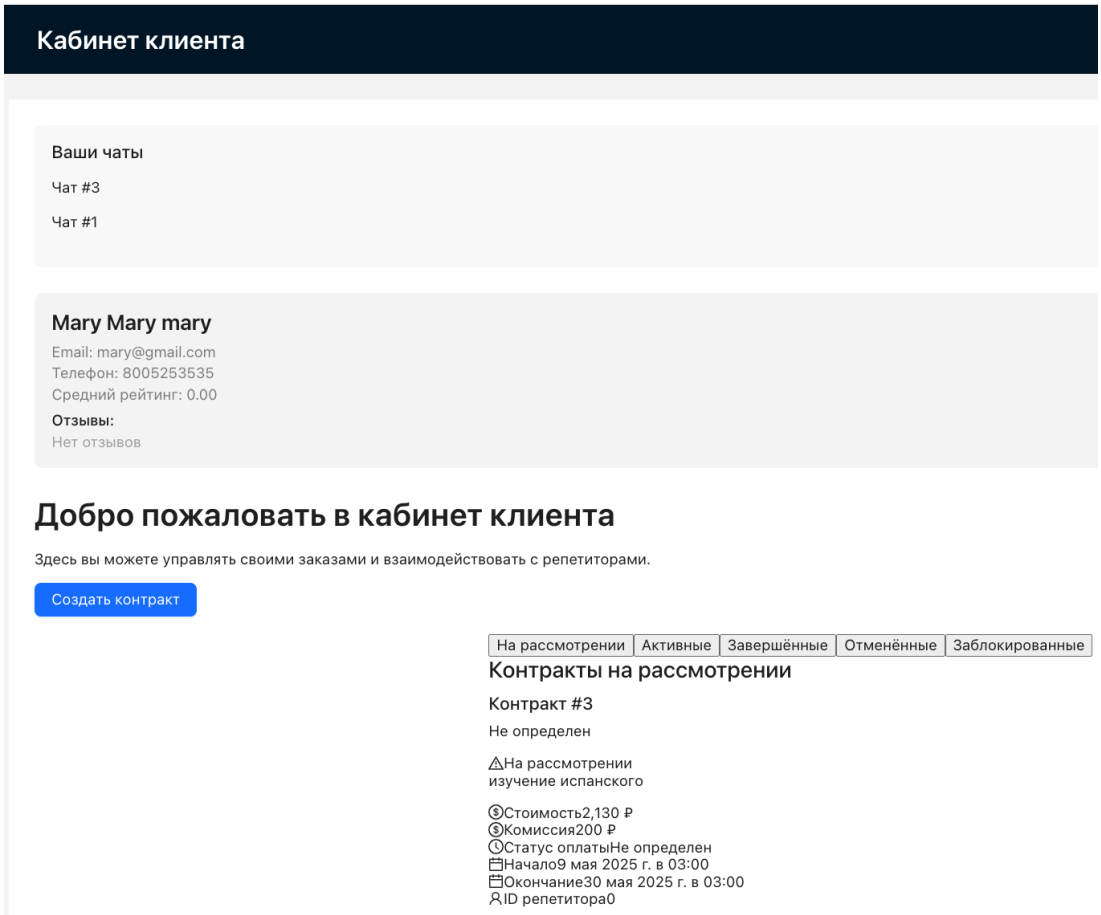

Рисунок 10 – Кабинет клиента

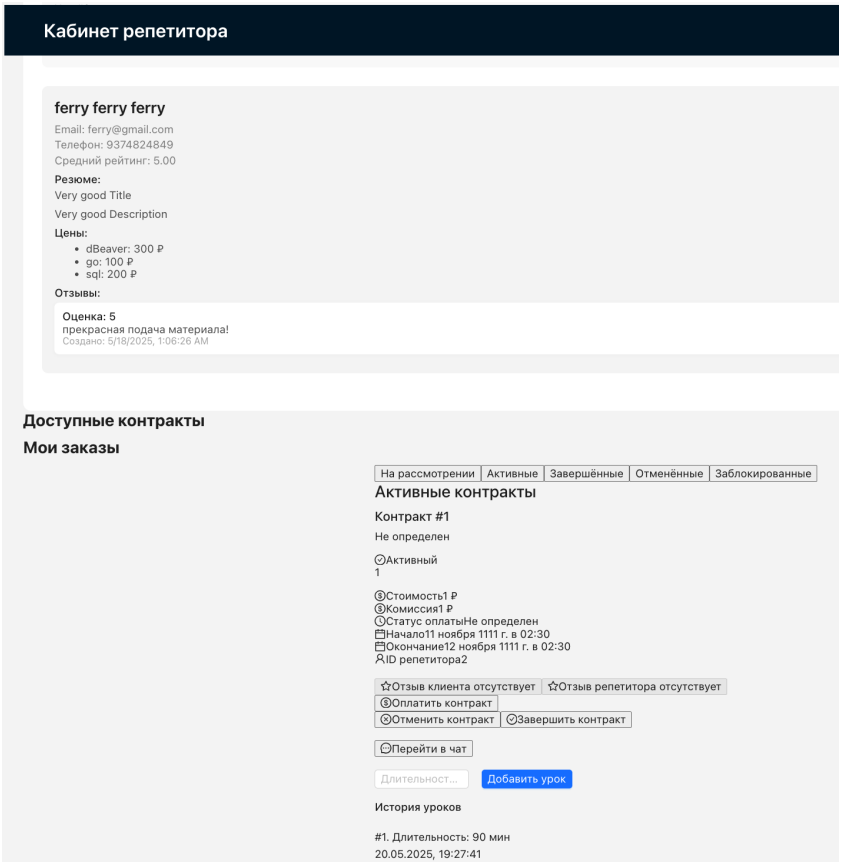

Рисунок 11 – Кабинет репетитора

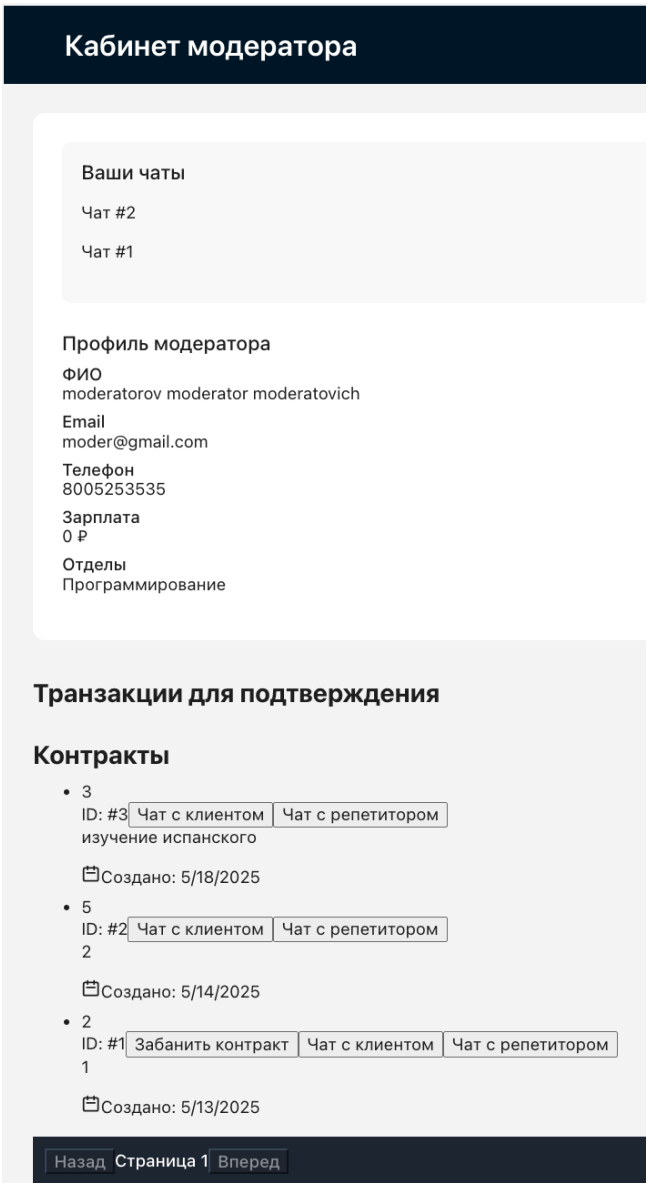

Рисунок 12 – Кабинет модератора (подтверждение контрактов)

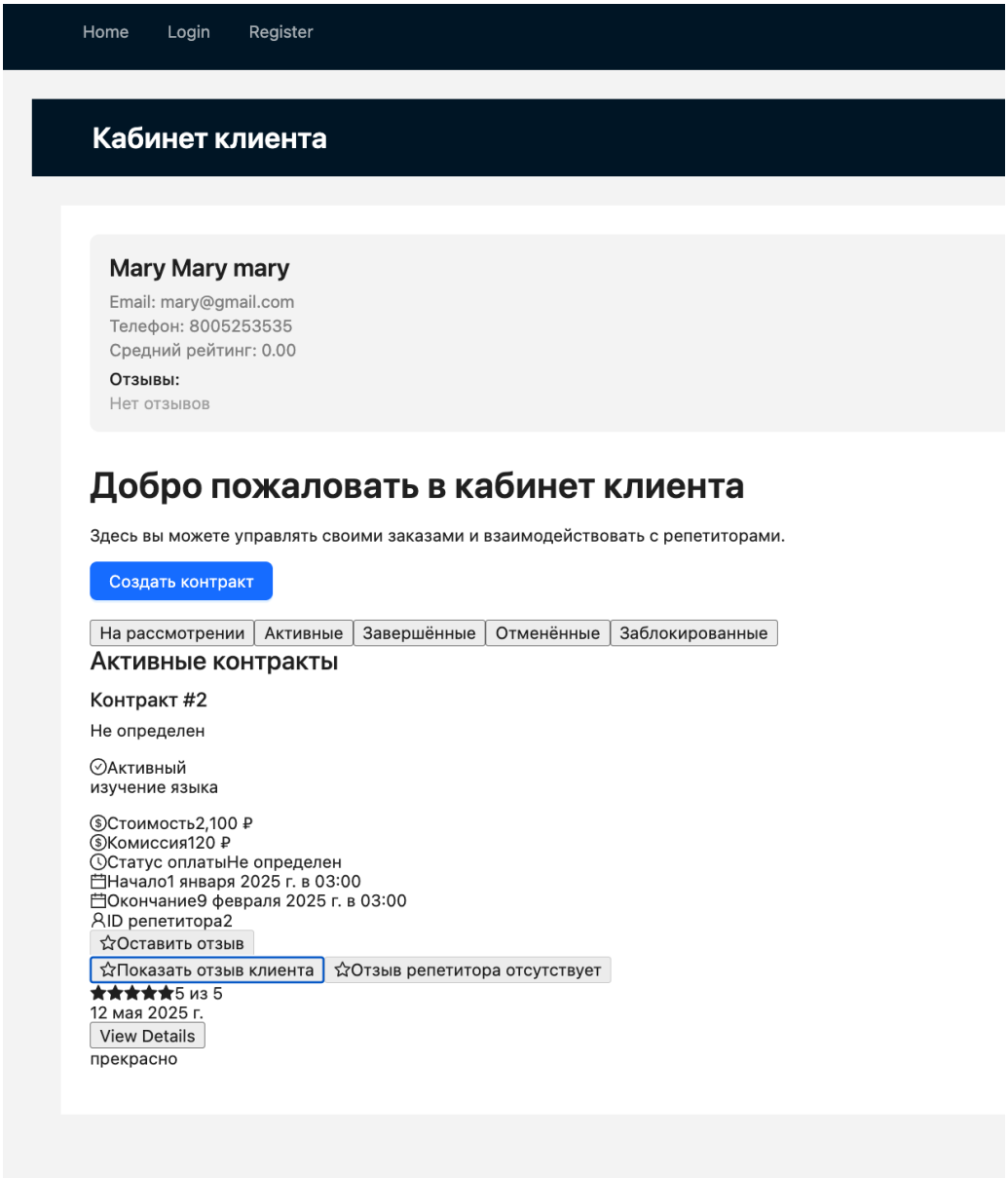

Рисунок 13 – Отзыв в кабинете клиента

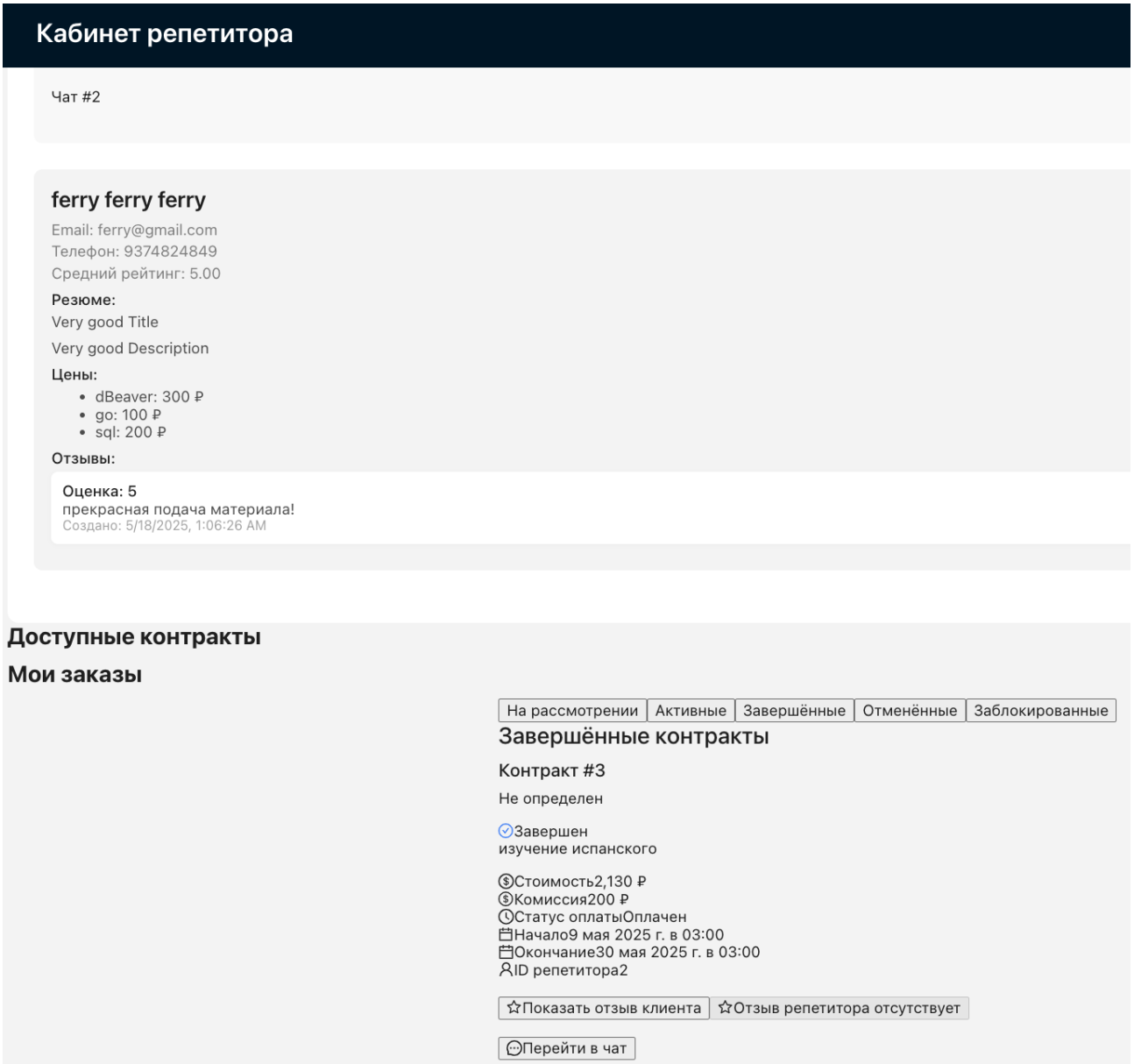

Рисунок 14 – Кабинет репетитора с завершенным заказом после одобрения транзакции модератором

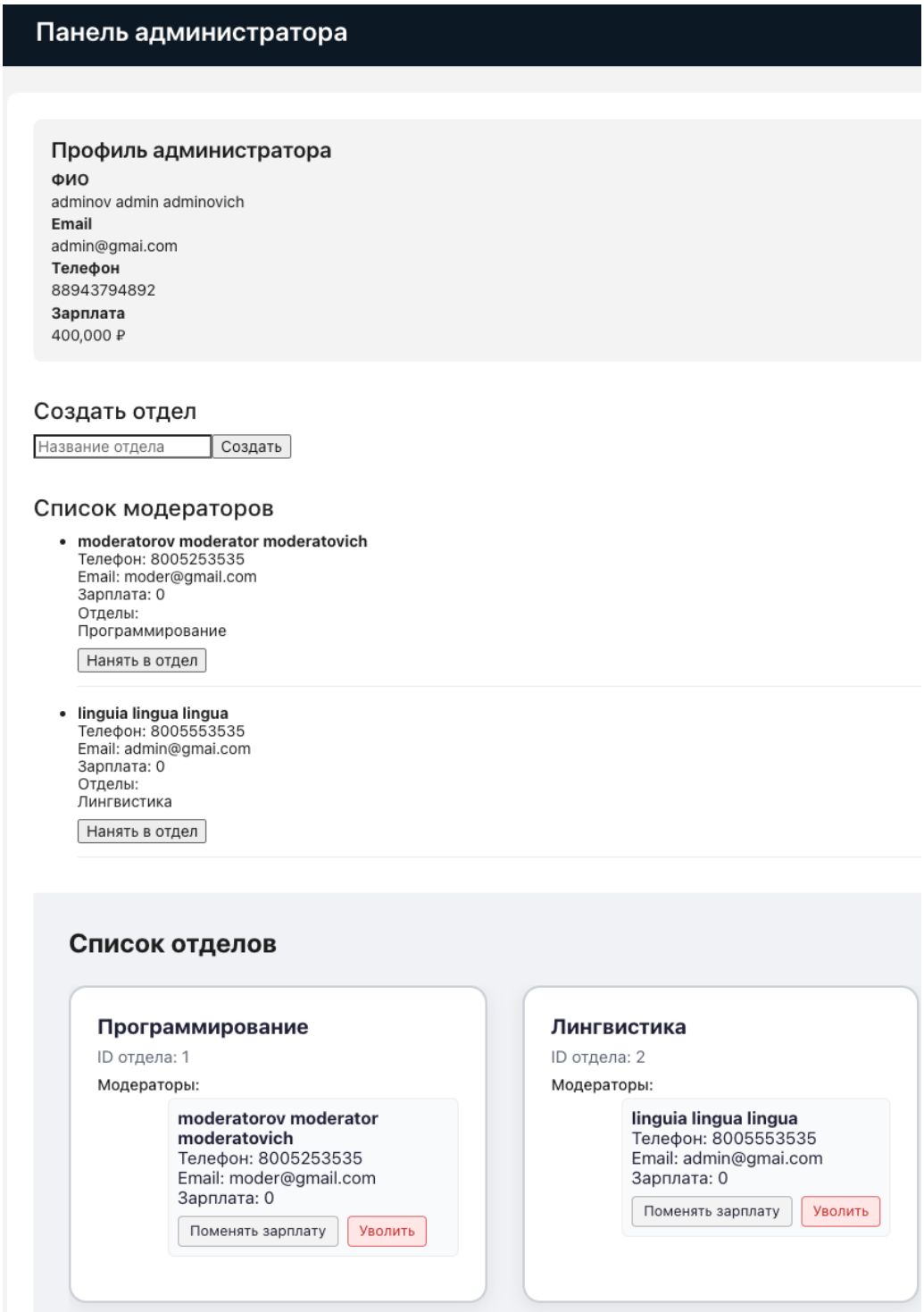

Рисунок 15 – Панель администратора

4 Исследовательская часть

4.1 Технические характеристики устройства

Технические характеристики используемого устройства [10]:

— операционная система macOS Monterey;

— оперативная память 8 ГБ;

— процессор Quad–Core Intel Core i5.

4.2 Влияние индексации на производительность запросов

Исследование проводилось с использованием случайно сгенерированных пользователей. В программе реализована функция GenerateUsers, создающая записи с уникальными значениями логина, электронной почты, номера телефона и пароля. Пользователь указывает необходимое количество записей через консольное меню.

Этапы исследования:

— **генерация данных** — с помощью функции GenerateUsers создаются записи пользователей, данные заносятся в базу данных.

— **замер времени запросов без индекса**, выполняются авторизационные запросы для каждого пользователя, и измеряется общее время выполнения.

— **создание индекса** — на поле login таблицы auth создается индекс, ускоряющий выполнение запросов по логину.

— **повторный замер с индексом** —выполняются те же запросы, и сравниваются значения времени.

В листингах 12–21 представлены результаты тестирования производительности системы при авторизации пользователей. Для каждого запуска отображается время выполнения запроса до и после добавления индекса при различном количестве пользователей.

Листинг 12 – Пример тестирования с 1000 пользователями

```
Enter size of users:
1000
Time taken without index with 1000 users: 120 ms
Time taken with index with 1000 users: 90 ms
Main menu
```

Листинг 13 – Пример тестирования с 2000 пользователями

```
Enter size of users:
2000
Time taken without index with 2000 users: 257 ms
Time taken with index with 2000 users: 145 ms
Main menu
```

Листинг 14 – Пример тестирования с 3000 пользователями

```
Enter size of users:
3000
Time taken without index with 3000 users: 583 ms
Time taken with index with 3000 users: 164 ms
Main menu
```

Листинг 15 – Пример тестирования с 4000 пользователями

```
Enter size of users:
4000
Time taken without index with 4000 users: 1094 ms
Time taken with index with 4000 users: 294 ms
Main menu
```

Листинг 16 – Пример тестирования с 5000 пользователями

```
Enter size of users:
5000
Time taken without index with 5000 users: 1522 ms
Time taken with index with 5000 users: 315 ms
Main menu
```

Листинг 17 – Пример тестирования с 6000 пользователями

```
Enter size of users:
6000
Time taken without index with 6000 users: 2250 ms
```

```
Time taken with index with 6000 users: 365 ms
Main menu
```

Листинг 18 – Пример тестирования с 7000 пользователями

```
Enter size of users:
7000
Time taken without index with 7000 users: 3161 ms
Time taken with index with 7000 users: 424 ms
Main menu
```

Листинг 19 – Пример тестирования с 8000 пользователями

```
Enter size of users:
8000
Time taken without index with 8000 users: 3729 ms
Time taken with index with 8000 users: 440 ms
Main menu
```

Листинг 20 – Пример тестирования с 9000 пользователями

```
Enter size of users:
9000
Time taken without index with 9000 users: 5010 ms
Time taken with index with 9000 users: 506 ms
Main menu
```

Листинг 21 – Пример тестирования с 10000 пользователями

```
Enter size of users:
10000
Time taken without index with 10000 users: 5782 ms
Time taken with index with 10000 users: 568 ms
Main menu
```

Результаты исследования представлены на рисунке 16.

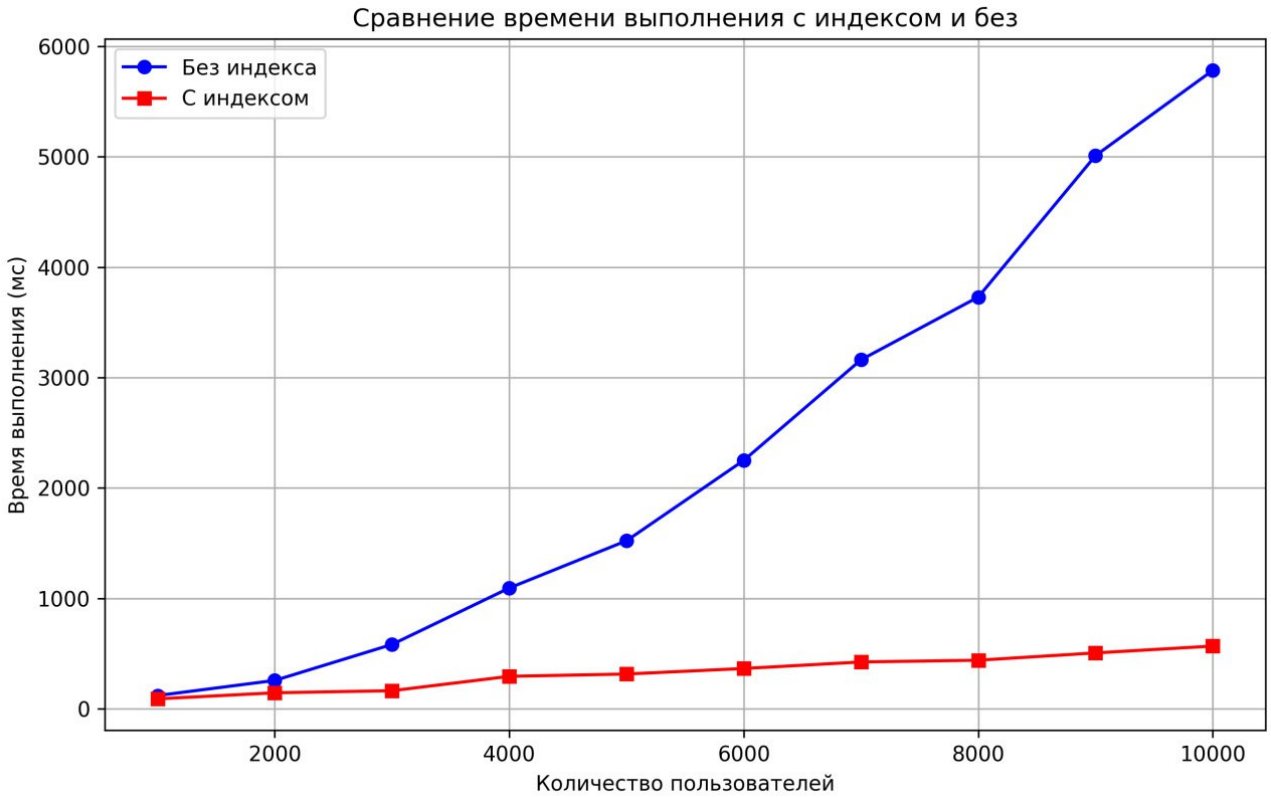

Рисунок 16 – Результаты исследования

4.3 Выводы

В результате проведенного исследования можно сделать следующие выводы:

— использование индексов в базе данных, таких как индекс по полю login в таблице auth, значительно ускоряет выполнение запросов при авторизации;

— при небольшом числе записей прирост производительности может быть незначительным, но с увеличением объема данных эффект становится более выраженным.

Таким образом, добавление индекса на поле, активно участвующее в запросах (например, login), позволяет повысить производительность системы при работе с большими объемами данных.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы была разработана база данных для управления репетиторскими услугами. Все поставленные задачи были успешно решены:

— проанализированы существующие методы и подходы к проектированию баз данных в области репетиторских услуг, выбраны оптимальные сущности и их взаимосвязи для эффективного хранения и обработки данных;

— спроектированы основные сущности базы данных, такие как информация о репетиторах, клиентах, занятиях и оплатах, а также установлены соответствующие ограничения целостности данных;

— реализованы механизмы автоматической генерации записей об оплатах при добавлении информации о занятиях, что значительно упрощает и автоматизирует процесс учета финансовых операций;

— разработана ролевая модель для разграничения доступа к данным в системе, что обеспечивает безопасность и удобство работы для пользователей с различными уровнями доступа;

— проведено тестирование функциональности базы данных, включая методы проверки корректности работы приложения, что подтверждает высокую степень работоспособности и стабильности системы;

— выполнено исследование производительности базы данных при увеличении объема данных и числа одновременно выполняемых запросов.

Результаты исследования показали, что производительность базы данных и приложения зависит от объема данных и числа одновременно выполняемых запросов. С увеличением числа записей в базе данных применение индексов также способствует ускорению выполнения запросов, что позволяет обеспечить эффективную работу системы даже при возросшей нагрузке.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Skyeng — онлайн-школа [Электронный ресурс]. Режим доступа: https://skyeng.ru. (дата обращения: 21.03.2025).
2. Profi.ru — сервис поиска специалистов [Электронный ресурс]. Режим доступа: https://profi.ru. (дата обращения: 21.03.2025).
3. TutorOnline — платформа для онлайн-репетиторства [Электронный ресурс]. Режим доступа: https://www.tutoronline.ru/. (дата обращения: 21.03.2025).
4. Preply — международная площадка для поиска репетиторов [Электронный ресурс]. Режим доступа: https://preply.com. (дата обращения: 21.03.2025).
5. В. Павлов Д. Триггеры в PL/SQL // Аллея науки. 2016. № 4. С. 759–761.
6. Group The PostgreSQL Global Development. PostgreSQL Documentation [Электронный ресурс]. Режим доступа: https://www.postgresql.org/docs/. 2025. (дата обращения: 08.05.2025).
7. DBeaver – универсальный инструмент для работы с базами данных [Электронный ресурс]. Режим доступа: https://dbeaver.io/. (дата обращения: 15.05.2025).
8. Организация интеграционного тестирования в задачах системного программирования / Л.Х. Сафиуллина, Ю.В. Торкунова, И.Р. Исмагилов [и др.] // Вестник Казанского технологического университета. 2018. Т. 16, № 2. С. 178–188.
9. Documentation P. Trigger Functions [Электронный ресурс]. Режим доступа: https://www.postgresql.org/docs/current/plpgsql-trigger.html. 2025. (дата обращения: 08.05.2025).
10. Support Apple. Как работает семейный доступ и общий доступ к подпискам [Электронный ресурс]. Режим доступа: https://support.apple.com/ru-ru/111339. 2025. (дата обращения: 08.05.2025).

ПРИЛОЖЕНИЕ А

Презентация к расчётно-пояснительной записке содержит 13 слайдов.

ПРИЛОЖЕНИЕ В

На листинге 22 представлен фрагмент кода, иллюстрирующий создание основных таблиц и ограничений целостности в базе данных.

Листинг 22 – Создание таблиц и ограничений базы данных

```
CREATE TABLE IF NOT EXISTS user_table (
    id SERIAL PRIMARY KEY,
    registration_date TIMESTAMP NOT NULL,
    last_login_date TIMESTAMP NOT NULL,
    personal_data_id INTEGER NOT NULL,
    FOREIGN KEY (personal_data_id) REFERENCES personal_data(id)
);
CREATE TABLE IF NOT EXISTS personal_data (
    id SERIAL PRIMARY KEY,
    telephone_number VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL,
    passport_number VARCHAR(255) NOT NULL,
    passport_series VARCHAR(255) NOT NULL,
    passport_date TIMESTAMP NOT NULL,
    passport_issued_by VARCHAR(255) NOT NULL,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    middle_name VARCHAR(255) NOT NULL
);
CREATE TABLE IF NOT EXISTS auth (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    user_type INTEGER NOT NULL,
    login VARCHAR(255) NOT NULL,
    password VARCHAR(255) NOT NULL,
    FOREIGN KEY (user_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS department (
    id SERIAL PRIMARY KEY,
    name VARCHAR(255) NOT NULL,
    head_id INTEGER NOT NULL
);
CREATE TABLE IF NOT EXISTS hire_info (
    id SERIAL PRIMARY KEY,
    department_id INTEGER NOT NULL,
```

```
    user_id INTEGER NOT NULL,
    FOREIGN KEY (department_id) REFERENCES department(id),
    FOREIGN KEY (user_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS admin (
    id SERIAL PRIMARY KEY,
    department_id INTEGER NOT NULL,
    salary INTEGER NOT NULL,
    FOREIGN KEY (id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS moderator (
    id SERIAL PRIMARY KEY,
    salary INTEGER NOT NULL,
    FOREIGN KEY (id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS client (
    id SERIAL PRIMARY KEY,
    FOREIGN KEY (id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS repetitor (
    id SERIAL PRIMARY KEY,
    resume_id INTEGER NOT NULL,
    summary_rating INTEGER NOT NULL,
    reviews_count INTEGER NOT NULL,
    FOREIGN KEY (id) REFERENCES user_table(id),
    FOREIGN KEY (resume_id) REFERENCES resume(id)
);
CREATE TABLE IF NOT EXISTS resume (
    id SERIAL PRIMARY KEY,
    repetitor_id INTEGER NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    prices JSONB NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    FOREIGN KEY (repetitor_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS chat (
    id SERIAL PRIMARY KEY,
    client_id INTEGER NOT NULL,
    repetitor_id INTEGER NOT NULL,
```

```
    moderator_id INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL
);
CREATE TABLE IF NOT EXISTS message (
    id SERIAL PRIMARY KEY,
    chat_id INTEGER NOT NULL,
    sender_id INTEGER NOT NULL,
    content TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL,
    FOREIGN KEY (chat_id) REFERENCES chat(id),
    FOREIGN KEY (sender_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS review (
    id SERIAL PRIMARY KEY,
    client_id INTEGER NOT NULL,
    repetitor_id INTEGER NOT NULL,
    rating INTEGER NOT NULL,
    comment TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL,
    FOREIGN KEY (client_id) REFERENCES user_table(id),
    FOREIGN KEY (repetitor_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS transaction (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    amount INTEGER NOT NULL,
    status INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL,
    type INTEGER NOT NULL,
    FOREIGN KEY (user_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS pending_contract_payment_transactions (
    id SERIAL PRIMARY KEY,
    user_id INTEGER NOT NULL,
    amount INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL,
    FOREIGN KEY (user_id) REFERENCES user_table(id)
);
CREATE TABLE IF NOT EXISTS contract (
    id SERIAL PRIMARY KEY,
    client_id INTEGER NOT NULL,
```

```
118     repetitor_id INTEGER NOT NULL,
119     review_client_id INTEGER NOT NULL,
120     review_repetitor_id INTEGER NOT NULL,
121     transaction_id INTEGER NOT NULL,
122     created_at TIMESTAMP NOT NULL,
123     description TEXT NOT NULL,
124     status INTEGER NOT NULL,
125     payment_status INTEGER NOT NULL,
126     price INTEGER NOT NULL,
127     commission INTEGER NOT NULL,
128     start_date TIMESTAMP NOT NULL,
129     end_date TIMESTAMP NOT NULL,
130     FOREIGN KEY (client_id) REFERENCES user_table(id)
131 );
```

В листинге 23 приведён пример команд SQL, назначающих права доступа к таблицам базы данных различным ролям.

Листинг 23 – Назначение прав доступа к таблицам базы данных

```
1  GRANT SELECT ON personal_data TO moderator, admin;
2  GRANT INSERT ON personal_data TO moderator, admin;
3  GRANT UPDATE ON personal_data TO moderator, admin;
4  GRANT DELETE ON personal_data TO admin;
5
6  GRANT SELECT ON "user" TO moderator, admin;
7  GRANT INSERT ON "user" TO moderator, admin;
8  GRANT UPDATE ON "user" TO moderator, admin;
9  GRANT DELETE ON "user" TO admin;
10
11 GRANT SELECT ON auth TO admin;
12 GRANT INSERT ON auth TO admin;
13 GRANT UPDATE ON auth TO admin;
14 GRANT DELETE ON auth TO admin;
15
16 GRANT SELECT ON chat TO client, repetitor, moderator, admin;
17 GRANT INSERT ON chat TO moderator, admin;
18 GRANT UPDATE ON chat TO client, repetitor, moderator, admin;
19 GRANT DELETE ON chat TO moderator, admin;
20
21 GRANT SELECT ON message TO client, repetitor, moderator, admin;
22 GRANT INSERT ON message TO client, repetitor, moderator, admin;
23 GRANT UPDATE ON message TO client, repetitor, moderator, admin;
```

```
GRANT DELETE ON message TO moderator, admin;

GRANT SELECT ON department TO moderator, admin;
GRANT INSERT ON department TO admin;
GRANT UPDATE ON department TO admin;
GRANT DELETE ON department TO admin;

GRANT SELECT ON client TO client, repetitor, moderator, admin;
GRANT INSERT ON client TO client, moderator, admin;
GRANT UPDATE ON client TO client, moderator, admin;
GRANT DELETE ON client TO admin;

GRANT SELECT ON resume TO client, repetitor, moderator, admin;
GRANT INSERT ON resume TO repetitor, moderator, admin;
GRANT UPDATE ON resume TO repetitor, moderator, admin;
GRANT DELETE ON resume TO admin;

GRANT SELECT ON review TO client, repetitor, moderator, admin;
GRANT INSERT ON review TO client, repetitor, moderator, admin;
GRANT UPDATE ON review TO client, repetitor, moderator, admin;
GRANT DELETE ON review TO moderator, admin;

GRANT SELECT ON repetitor TO guest, client, repetitor, moderator, admin;
GRANT INSERT ON repetitor TO repetitor, moderator, admin;
GRANT UPDATE ON repetitor TO repetitor, moderator, admin;
GRANT DELETE ON repetitor TO admin;

GRANT SELECT ON contract TO client, repetitor, moderator, admin;
GRANT INSERT ON contract TO client, repetitor, moderator, admin;
GRANT UPDATE ON contract TO moderator, admin;
GRANT DELETE ON contract TO admin;

GRANT SELECT ON admin TO admin;
GRANT INSERT ON admin TO admin;
GRANT UPDATE ON admin TO admin;
GRANT DELETE ON admin TO admin;

GRANT SELECT ON moderator TO moderator, admin;
GRANT INSERT ON moderator TO admin;
GRANT UPDATE ON moderator TO admin;
GRANT DELETE ON moderator TO admin;
```

```

GRANT SELECT ON "user" TO client, repetitor, moderator, admin;
GRANT INSERT ON "user" TO admin;
GRANT UPDATE ON "user" TO moderator, admin;
GRANT DELETE ON "user" TO admin;

GRANT SELECT ON transaction TO client, repetitor, moderator, admin;
GRANT INSERT ON transaction TO client, repetitor, moderator, admin;
GRANT UPDATE ON transaction TO moderator, admin;

GRANT DELETE ON transaction TO admin;

GRANT SELECT ON pending_contract_payment_transactions TO moderator, admin;
GRANT INSERT ON pending_contract_payment_transactions TO moderator, admin;
GRANT UPDATE ON pending_contract_payment_transactions TO moderator, admin;
GRANT DELETE ON pending_contract_payment_transactions TO moderator, admin;

GRANT SELECT ON lesson TO client, repetitor, moderator, admin;
GRANT INSERT ON lesson TO client, repetitor, moderator, admin;
GRANT UPDATE ON lesson TO client, repetitor, moderator, admin;
GRANT DELETE ON lesson TO admin;
```